

Asynchronous Distributed Key Generation with Flexible Threshold and Batched Key Derivation

Junming Li, Renfei Shen, Zhi Lu, Songfeng Lu, Xueming Tang, Yuanqing Feng

Abstract—Distributed Key Generation (DKG) is a vital security foundation for decentralized technologies such as threshold encryption and signatures. However, existing asynchronous DKG (ADKG) protocols still suffer from rigid threshold configuration, limited throughput, and poor scalability. In this paper, we propose an efficient ADKG framework that support flexible threshold. First, we design a Flexible-Threshold Asynchronous Complete Secret Sharing (FT-ACSS) protocol based on asymmetric bivariate polynomials and a KZG-based polynomial commitment scheme. In an $n \geq 3f + 1$ asynchronous network, FT-ACSS allows the dealer to flexibly choose the reconstruction threshold $t \in [f, n - f]$, surpassing the limitations of traditional fixed f or $2f$ thresholds. Furthermore, FT-ACSS supports a packed secret sharing to share and reconstruct up to t secrets with $O(\lambda n^2)$ communication complexity. We additionally extend the classical Schnorr protocol into a non-interactive m -Schnorr that supports proofs for multiple secrets, significantly reducing amortized resource overhead. Combined with RBC and ABA, these components form a complete Flexible-Threshold Asynchronous Distributed Key Generation (FT-ADKG) protocol. Theoretical analysis and experimental results show that FT-ADKG achieves stronger threshold flexibility and lower amortized overhead than state-of-the-art ADKG protocols, making it well suited to large-scale asynchronous distributed systems.

Index Terms—Asynchronous distributed key generation, asynchronous complete secret sharing, network security, distributed systems, non-interactive zero-knowledge proofs, threshold cryptography, public key.

I. INTRODUCTION

The Distributed Key Generation (DKG) protocol enables a group of untrusted nodes to jointly generate public-private key pairs, without the need of a trusted third party. The public key is shared internally within the system, while the private key is distributed among the nodes via a threshold secret sharing scheme that prevents individual entities from reconstructing or storing the full private key. Under this mechanism, no single entity, nor any adversary controlling fewer than a threshold number of nodes, can obtain, reconstruct, or leak the full private key, thereby fundamentally eliminating the risk of single-point failures and bottlenecks caused by centralized trust in the system.

The private key established through a DKG protocol serves as a fundamental building block for dependable and secure distributed systems. First, it enables the deployment of threshold cryptographic services, such as threshold signatures [1], [2] and threshold encryption [3]–[6], which eliminate single points of trust in key usage. Second, in Byzantine fault-tolerant (BFT)

consensus systems [7]–[12], DKG is commonly used to generate common coins, thereby supporting the liveness and reliable progress of consensus protocols under adversarial conditions. Third, in distributed ledger and blockchain infrastructures, DKG provides a practical trust foundation for secure wallet management [13], committee formation [14], and random beacon generation [15]. In addition, DKG is also relevant to large-scale IoT and edge-cloud environments, where it can support device authentication [16], distributed data protection [17], and collaborative key management across heterogeneous nodes [18]. These applications highlight the importance of efficient and robust DKG protocols as a core infrastructure primitive for modern distributed security services.

Secret sharing [19], [20] is the foundation of Distributed Key Generation, which divides the original secret into multiple fragments and distributes them to other nodes. The original secret can be reconstructed only if more than the reconstruction threshold number of valid shares have been collected. There are many functional enhancements of secret sharing, including Verifiable Secret Sharing [21], Complete Secret Sharing [22], [23], and Packed Secret Sharing [24].

Early DKG protocols were developed under a relatively idealized synchronous network model [2], [6], [25]–[29], where message delays are assumed to be bounded and predictable. However, as network environments become increasingly complex, the diversity of parties' behavior and the unpredictability of network fluctuations make such synchronous assumptions difficult to satisfy in practice. Consequently, recent studies have shifted toward Asynchronous Distributed Key Generation (ADKG) [30]–[37]. In the asynchronous setting, protocols do not rely on a global clock or any known upper bound on message delays, and can therefore tolerate arbitrary and unpredictable communication latency. Existing ADKG protocols typically ensure liveness and security under such conditions by combining primitives such as Asynchronous Verifiable Secret Sharing (AVSS) [5], [38]–[40], Asynchronous Complete Secret Sharing (ACSS) [23], [34], [35], Reliable Broadcast (RBC) [34], [41], and Asynchronous Binary Agreement (ABA). In this paper, we focus on the design of high-performance ACSS and ADKG protocols.

Kokoris et al. [30] proposed the first ADKG protocol, achieving a total communication cost of $O(\lambda n^4)$ and an average round complexity of $O(n)$ by instantiating n parallel high-threshold ACSS instances, where λ is a security parameter. Abraham et al. [31] reduced the communication overhead to $O(\lambda n^3 \log n)$. Gao et al. [42] improved upon Abraham et al.'s construction scheme [31], achieving an improved communication complexity of $O(\lambda n^3)$. However, the private

TABLE I
COMPARISON TABLE OF RELEVANT SECRET SHARING SCHEMES.

Protocol	Communication Cost	Trust Setup	High Threshold	Assumptions
Cachin et al. [5]	$O(\lambda n^3)$	-	✓	DL
hbACSS [23]	$O(\lambda n^2 \log n)$	✓	×	q -SDH [43]
Kokoris et al. [30]	$O(\lambda n^3)$	-	✓	DDH
Backes et al. [44]	$O(\lambda n^2)$	✓	×	q -SDH, q -polyDH
Shoup et al. [45]	$O(\lambda n^2 \log n)$	-	×	Random Beacon [†]
Haven [40]	$O(\lambda n^2 \log n)$	-	✓	DL
Das et al. [34]	$O(\lambda n^2)$	-	✓	DDH
Haven++ [46]	$O(\lambda n^2 \log n)$	-	✓	DL
Das et al. [35]	$O(\lambda n^2)$	-	✓	DL
Bingo [36]	$O(\lambda n^2)$	✓	✓	q -SDH [43]
HARTS [37]	$O(\lambda n^2 \log n)$	-	✓	OMDL
FT-ACSS (our work)	$O(\lambda n^2)$	✓	✓	q -SDH

[†] Shoup et al. [45] relies on lightweight cryptographic assumptions, and they also present a simpler variant in the ROM.

TABLE II
COMPARISON TABLE OF RELEVANT (A)DKG SCHEMES.

Protocol	Net.	Fault Toler.	Commun.	Round	High Thresh.	Trusted Setup	Flexible Thresh.	Batched Key Generation	Crypto Assump.	Setup Assump.
Gennaro et al. [2]	sync.	1/2	$O(n \cdot \mathcal{B}(\lambda))$	$O(\mathcal{R})$	✓	✓	×	×	DDH	-
Canetti et al. [27]	sync.	1/2	$O(n \cdot \mathcal{B}(\lambda))$	$O(\mathcal{R})$	✓	✓	×	×	DDH	-
Neji et al. [28]	sync.	1/2	$O(n \cdot \mathcal{B}(\lambda))$	$O(\mathcal{R})$	✓	✓	×	×	DDH	-
Kokoris et al. [30]	async.	1/3	$O(\lambda n^4)$	$O(n)$	✓	-	×	×	DDH	RO
Abraham et al. [31]	async.	1/3	$O(\lambda n^3)$	$O(1)$	×	-	×	×	SXDH	RO&PKI
Das et al. [34]	async.	1/3	$O(\lambda n^3)$	$O(\log n)$	✓	-	×	×	DDH	RO&PKI
Groth et al. [33]	async.	1/3	$O(\lambda n^3)$	$O(1)$	×	-	×	×	DL	RO&PKI
Das et al. [35]	async.	1/3	$O(\lambda n^3)$	$O(\log n)$	✓	-	×	×	DL	RO&PKI
Bingo [36]	async.	1/3	$O(\lambda n^3)$	$O(1)$	✓	✓	✓	×	q -SDH	PKI
HARTS [37]	async.	1/3	$O(\lambda n^3 \log n)$	$O(1)$	✓	-	✓	×	DL	RO&PKI
FT-ADKG (our work)	async.	1/3	$O(\lambda n^3)$	$O(1)$	✓	✓	✓	✓	q -SDH	RO&PKI

¹ We use $\mathcal{B}(\lambda)$ and $\mathcal{R}$ to denote the communication cost of Byzantine broadcast of λ -bit message and round complexity of Byzantine broadcast, respectively.

² **Abbreviation:** Net. denotes network model, with sync. and async. referring to synchronous and asynchronous. Fault Toler. denotes the maximum number of malicious nodes tolerated by the network. Commun. denotes communication cost. Thresh. denotes the reconstruction threshold. Assump. denotes the cryptographic or setup assumptions.

³ All the above protocols generate secret keys as field elements, and are therefore readily compatible with mainstream threshold encryption and threshold signature schemes.

key generated by this scheme is a group element $g \in \mathbb{G}$ rather than a field element $z \in \mathbb{F}$, making it difficult to be compatible with traditional threshold cryptosystems [1]–[6]. Das et al. [34] established consensus by deploying n parallel ABA instances, maintaining $O(\lambda n^3)$ communication overhead, but requiring $O(\log n)$ rounds of communication. Groth and Shoup [33] avoided trusted setups while maintaining $O(\lambda n^3)$ communication complexity. However, their scheme is limited to low-threshold scenarios and lacks support for high-threshold reconstruction. We compare the most relevant Secret Sharing and DKG schemes in Tables I and II.

Existing solutions are limited by rigid threshold configurations, low throughput, high amortized overhead, limited key compatibility, and idealized network assumptions, making it difficult to meet the security management needs of large-scale dynamic distributed systems. First, we propose an efficient asynchronous complete secret sharing (FT-ACSS) protocol that supports flexible threshold configuration, enabling parallel batch sharing of m secrets. Second, we extend the Schnorr protocol, proposing a non-interactive m -Schnorr protocol adapted

to FT-ACSS, which can simultaneously generate commitments for m secrets, thus significantly improving communication efficiency. Third, by jointly leveraging Reliable Broadcast (RBC) and Asynchronous Binary Agreement (ABA) primitives, we integrate these technical modules into a complete FT-ADKG protocol. We present the overall workflow of the proposed protocol in Fig.1.

In an asynchronous network consisting of $n \geq 3f + 1$ nodes (containing at most f malicious nodes), the FT-ACSS protocol achieves a communication complexity of $O(\lambda n^2)$, while FT-ADKG achieves $O(\lambda n^3)$. The overall security of the scheme is built upon the q -SDH assumption and the random oracle model (ROM), and requires PKI. The contributions are as follows:

- **Flexible Threshold Configuration:** We implement a configurable reconstruction threshold for FT-ACSS over the range $t \in [f, n - f]$ by adjusting the construction of the secret-sharing polynomial. This feature enables system designers to flexibly balance system availability and security strength according to the requirements of different

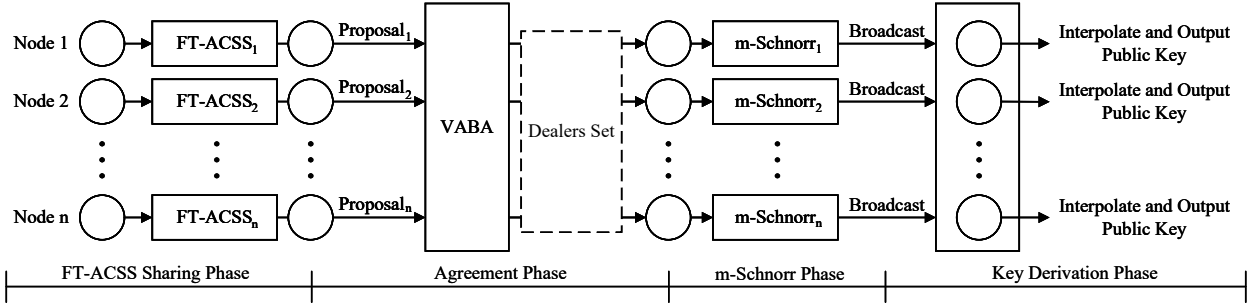


Fig. 1. In an asynchronous network consisting of n nodes, we present an overview of the proposed FT-ADKG protocol. During the secret sharing phase, each node initiates an FT-ACSS instance to distribute a random secret. After the completion of FT-ACSS, the protocol enters the agreement phase, where each node takes a proposal set of candidate dealers as input to invoke the VABA protocol. As a result, all honest nodes eventually reach agreement on a common Dealers Set. Given this agreed Dealers Set, each node computes its local private-key share accordingly. It then executes the m-Schnorr protocol to generate the corresponding public-key share and proof, which are broadcast to the entire network. In the key derivation phase, each node verifies the public-key shares received from other nodes. Once at least $f + 1$ valid shares have been collected, the final public key for this execution can be reconstructed through interpolation.

services, thereby significantly improving the adaptability of distributed system design.

- **Efficient Distribution and Reconstruction:** To improve communication efficiency in asynchronous environments, we extend the traditional univariate polynomial to a bivariate asymmetric polynomial $\Phi(X, Y)$. The X -dimension is used to conceal and distribute the core secret, while the Y -dimension enables decentralized acceleration of the sharing process. This design allows all nodes to contribute equally to the secret distribution and reconstruction, effectively avoiding protocol blocking due to single-point performance bottlenecks or failures, and significantly improving the robustness and operational efficiency of the system.
- **Lightweight Verification:** In asynchronous networks, to prevent malicious nodes from injecting invalid shares or tampering with existing ones, most existing schemes rely on polynomial commitment schemes (PCS) to generate share proofs. By introducing HashToCurve [52] and adapting it for FT-ACSS, we develop a lightweight KZG-based verification scheme built on the Bingo protocol. This scheme preserves the efficiency and simplicity of the KZG [43] protocol while ensuring resistance against malicious nodes. This design effectively reduces the computational and communication overhead of nodes during the sharing verification process, which improves the reliability of the protocol in large-scale network environments, especially in resource-constrained environments.
- **Batch Secret Sharing:** Unlike traditional schemes that require repeated executions, FT-ACSS supports sharing $m \in [1, t + 1]$ secrets within a single parallel execution without increasing additional communication complexity. Combined with the m-Schnorr protocol, FT-ADKG can generate m corresponding keys in a single execution. This significantly reduces the amortized overhead of the system and provides excellent throughput performance for high-frequency key request scenarios.
- **High Compatibility:** All private key shares generated by FT-ADKG are field elements $sk \in \mathbb{F}_q$. This property ensures seamless compatibility with existing threshold cryptosys-

tems based on the discrete logarithm assumption (such as threshold signatures, threshold encryption, etc.), greatly reducing the integration difficulty of the protocol into existing decentralized infrastructures.

II. RELATED WORK

In this section, we focus on the lines of work most relevant to our setting, namely AVSS and ACSS. Although VSS in synchronous networks is foundational to the area, it is not the main focus of our comparison.

Secret Sharing. Cachin et al. [5] proposed a AVSS protocol with a communication complexity of $O(\lambda n^3)$, based on DL assumption. Yurek et al. [23] improved efficiency via batching and packing together with more communication-efficient verification, reducing communication to near-quadratic bounds such as $O(\lambda n^2 \log n)$. Kokoris-Kogias et al. [30] presented a high-threshold ACSS construction based on asymmetric bivariate polynomials with $O(\lambda n^3)$ communication. Backes et al. [44] improved Cachin et al. [5] by introducing an AVSS protocol with $O(\lambda n^2)$ communication complexity, though its recovery threshold is limited to f . Shoup et al. [45] proposed a lightweight AVSS protocol that avoids DL and bilinear-pairing assumptions, with a simpler ROM-based variant and $O(\lambda n^2 \log n)$ communication complexity, but it does not support high-threshold secret sharing. AlHaddad et al. [40] introduced Haven protocol, which achieves $O(\lambda n^2 \log n)$ communication complexity. Notably, Haven is the first high-threshold secret sharing scheme that does not rely on a PKI or trusted setup. Das et al. [34] further reduced the communication to $O(\lambda n^2)$ at the price of relying on ROM, PKI and non-standard assumptions. Alhaddad et al. proposed Haven++, which enables a dealer to share multiple secrets in a single sharing with communication complexity $O(\lambda n^2 \log n)$. Abraham et al. [36] introduced Bingo protocol, which has a total communication complexity of $O(\lambda n^2)$. Bingo is the first high-threshold AVSS protocol that is adaptively secure and can packed share up to $f + 1$ secrets. Kokoris-Kogias et al. [37] proposed HARTS, which achieves $O(\lambda n^2 \log n)$ communication, avoids trusted setup and bilinear pairings, and supports a flexible high-threshold setting.

Numerous works have examined the problem of Distributed Key Generation with various cryptographic assumptions, network model, secret sharing and with other properties. We categorize prior work into two classes based on network assumptions: *Synchronous* and *Asynchronous*.

Synchronous DKG. The synchronous network assumes relatively ideal conditions where all nodes have equal performance and resources, and messages are successfully delivered within an ideal time frame. Synchronous DKG has been studied for decades. Pedersen [25] proposed the first DKG scheme based on verifiable secret sharing [26]. Gennaro et al. [2] demonstrated that an attacker could bias the public key generated by this protocol and proposed a solution, though computationally expensive. Canetti et al. [27] and Neji et al. [28] improved Gennaro's method by introducing adaptive security and increasing computational efficiency. Gurkan et al. [6] designed a DKG protocol based on publicly verifiable secret sharing (PVSS), with a communication complexity of $O(n^3 \log n)$. However, the generated keys belong to group elements, which is a limitation. Shrestha et al. [29] proposed a protocol with $O(n^3)$ communication complexity that does not rely on broadcast channels. The GrandLine protocol implements a lightweight, adaptively secure, and efficient DKG scheme. However, the generated keys are group elements.

Asynchronous DKG. The asynchronous network is closer to real-world network conditions, where issues such as varying computational power across nodes and potential message loss or delivery failures due to various factors must be considered. Kokoris et al. [30] designed the first ADKG protocol, which has a communication complexity of $O(\lambda n^4)$ and a round complexity of $O(n)$. Abraham et al. [31] devised a consensus protocol and combined it with Gurkan et al.'s PVSS protocol [6] to construct a high-threshold ADKG with a communication complexity of $O(n^3 \log n)$. However, this protocol inherits the drawback of generating keys that belong to group elements, making it incompatible with widely-used threshold cryptography schemes [32] based on field elements. Groth and Shoup [33] proposed a DKG protocol that does not require a trusted setup and has a communication complexity of $O(n^3)$. Das et al. [34] developed a high-threshold DKG with a communication complexity of $O(\lambda n^3)$, and the generated keys belong to field elements, based on the DDH assumption. More recently, Das et al. [35] introduced an improved scheme based on DL assumption, which offers higher security than the DDH assumption. They also enhanced the efficiency of the high-threshold ADKG, making it nearly as efficient as low-threshold ADKG. Recently, Abraham et al. [36] proposed Bingo protocol that achieves adaptive security, with a communication complexity of $O(\lambda n^3)$ and a round complexity of $O(1)$. Bacho et al. [37] proposed HARTS, in which the PADKG protocol achieves $O(\lambda n^3 \log n)$ communication and $O(1)$ rounds without trusted setup. Both Bingo [36] and HARTS [37] support packed secret sharing and flexible thresholds.

III. PRELIMINARIES

This section presents the basic notation and reviews the definitions of threat model, reliable broadcast, secret sharing,

polynomial commitment scheme, zero-knowledge proof, and asynchronous Byzantine agreement.

A. Notations

TABLE III
NOTATIONS USED IN THE PAPER

Notation	Description
λ	Security parameter
n	Total number of nodes, where $n \geq 3f + 1$
f	Maximum number of malicious nodes
t	Reconstruction threshold of secret sharing
$\mathbb{F}_q$	Field of order q where q is prime
$\mathbb{G}$	Cyclic group of order q
$g, \hat{g}, h$	Random and independent generators of $\mathbb{G}$
z, g^z	ADKG secret and public key
m	Total number of secret, $m \in [1, t + 1]$
$\Phi(X, Y)$	Asymmetric bivariate polynomial
$z_k(i)$	k -th secret shared by the i -th node
$z(i)$	Set of secrets shared by the i -th node
$g^{z_k(i)}, g^{z(i)}$	i -th node's threshold public keys
pk_i, sk_i	Communication key pair of the i -th node
Com	NIZK commitment
PPT	Probabilistic Polynomial Time
$\mathcal{A}$	PPT adversaries
$\text{negl}(\cdot)$	A negligible probability function takes $\cdot$ as input
$ \mathbb{S} $	The size of $\mathbb{S}$

The symbols used in this paper are referenced in Table III. λ is the security parameter. For example, λ can denote the output size of a hash function. For a finite set $\mathbb{S}$, $|\mathbb{S}|$ denotes its size. The notation $x \xleftarrow{\mathbb{S}}$ denotes sampling a random element from the set $\mathbb{S}$ and assigning it to x . For any $n \in \mathbb{N}$, we define $[n] = \{1, \dots, n\}$ and $\llbracket n \rrbracket = \{0, \dots, n\}$. Specifically, for two integers $i < j$, we define the ordered set $[i, j] = \{i, \dots, j\}$.

B. Threat Model

We consider an asynchronous network of $n \geq 3f + 1$ nodes, in which every pair of nodes is connected via an authenticated point-to-point channel. Message delays are unbounded and unpredictable, and the nodes cannot rely on any global clock to infer the states of other nodes. We assume a malicious adversary $\mathcal{A}$ that can corrupt and control up to f nodes in the network. The adversary $\mathcal{A}$ can arbitrarily delay any messages during protocol, but must eventually deliver all messages sent between honest nodes.

C. Reliable Broadcast

Definition 1 (Reliable Broadcast (RBC) Protocol). RBC [22], [41] is a communication primitive that guarantees that all honest nodes in a distributed system eventually deliver the same message. It ensures reliable data dissemination provided that more than two-thirds of the nodes are honest. The protocol is typically defined by three properties: *Validity*, *Agreement*, and *Totality*.

- **Validity.** If the dealer is honest and its input is msg , then every honest party that completes the protocol outputs the dealer's input msg .
- **Agreement.** If one honest node outputs a message msg , and another honest node outputs a message msg' , then $msg' = msg$.

- **Totality.** If the dealer is honest, all honest parties eventually complete the protocol and output a value. Moreover, if any honest party completes the protocol and outputs a message msg , then all honest parties eventually complete the protocol and output the same message msg .

D. Secret Sharing

Definition 2 (Shamir Secret Sharing). Shamir secret sharing [19] realizes a (t, n) -threshold scheme over a finite field $\mathbb{F}_q$, where q is prime. To share a secret $s \in \mathbb{F}_q$, the dealer chooses a random polynomial $P(x) = s + a_1x + a_2x^2 + \dots + a_tx^t$ ($a_1, \dots, a_t \xleftarrow{\$} \mathbb{F}_q$) such that $P(0) = s$, and distributes shares $(x_i, P(x_i))$ for distinct nonzero $x_i \in \mathbb{F}_q$. Any $t + 1$ correct shares can reconstruct s by polynomial interpolation, while fewer than t shares reveal no information about the secret. Consequently, the scheme guarantees privacy against any adversary corrupting at most $f < t$ parties.

Definition 3 (Asynchronous Verifiable Secret Sharing (AVSS) Protocol). An AVSS is defined in an asynchronous Byzantine network. In such a protocol, dealer shares a secret among the parties, and the distributed shares are verifiable. AVSS protocol consists of two phases: the sharing phase Sh, and the reconstruction phase Rec. For any adversary $\mathcal{A}$ corrupting at most f nodes, the protocol satisfies the following properties with overwhelming probability of $1 - \text{negl}(\lambda)$ [5], [38]–[40].

- **Termination.** If the dealer is honest, then after the sharing phase, all honest nodes will obtain the correct secret shares. Accordingly, if all honest nodes execute the reconstruction phase, they will eventually reconstruct the original secret.
- **Privacy.** If the dealer is honest, an adversary $\mathcal{A}$ holding at most f secret shares cannot obtain any information about the original secret.
- **Agreement.** In the reconstruction phase, if shares held by the parties are valid, then all honest parties will be able to reconstruct the same original secret.

Definition 4 (Asynchronous Complete Secret Sharing (ACSS) Protocol). However, an AVSS is not enough since in the worst case t honest parties may not get their shares, while an ACSS protocol requires all honest parties to get their shares. ACSS extends AVSS by additionally guaranteeing that all honest parties eventually receive valid shares. In particular, ACSS satisfies the following property [22], [23]:

- **Completeness.** If any honest party terminates the sharing protocol, then there exists a polynomial $F(\cdot)$ of t -degree such that $F(0) = s'$, and every honest party eventually obtains a share $s_i = F(x_i)$. If the dealer is honest, then $s' = s$.

E. Polynomial Commitment Scheme

PCS [43], [47] is a cryptographic protocol that allows a dealer to generate a proof for a polynomial while keeping it hidden, and later prove the evaluation of the polynomial at a specific point without revealing the original polynomial. This is particularly useful in various cryptographic applications [48], including zero-knowledge proofs, verifiable computation,

and blockchain systems. A PCS consists of the following algorithms:

- $\text{CRS} \leftarrow \text{Setup}(1^\lambda)$ takes a security parameter λ as input and outputs a set of global parameters CRS.
- $\text{Com} \leftarrow \text{Commit}(\text{CRS}, \alpha(x))$ takes CRS and a polynomial α as input and computes the commitment Com.
- $p, \pi \leftarrow \text{Eval}(\text{CRS}, \alpha(X), x)$ takes CRS, a polynomial α , and a variable x as input. The output is $p = \alpha(x)$ along with a proof π at $(x, \alpha(x))$.
- $\text{True/False} \leftarrow \text{Verify}(\text{CRS}, \text{Com}, (x, p), \pi)$ takes CRS, Com, (x, p) , and a proof π as input. If (x, p) is a point on the polynomial, the verification passes and returns True; otherwise, it returns False.

Meanwhile, PCS also satisfies the following security properties: Binding, Hiding and Correctness. We will introduce it in Definition 5.

Definition 5 (Kate–Zaverucha–Goldberg (KZG) [43] Polynomial Commitment). We employ the KZG polynomial commitment scheme as the core verification tool in our construction. KZG is a pairing-based scheme with a trusted setup, which supports constant-size commitments and succinct evaluation proofs for bounded-degree polynomials. In our construction, KZG is used to provide lightweight proofs for secret polynomials and distributed shares, thereby reducing the verification overhead and improving the scalability of the ACSS/ADKG protocol in asynchronous networks. KZG PCS satisfy the following security properties [36]:

- **Evaluation Binding:** Consider a game $G_{\mathcal{A}}^{\text{eval-binding}}(\lambda)$. An adversary $\mathcal{A}$ takes public parameter λ as input and outputs $(\text{Com}, x, (p_1, p_2), (\pi_1, \pi_2))$. It wins if $\text{Verify}(\text{Com}, x, p_1, \pi_1) = \text{Verify}(\text{Com}, x, p_2, \pi_2) = \text{True}$ and $p_1 \neq p_2$. We say that the PCS satisfies evaluation binding if for every PPT adversary $\mathcal{A}$, the probability that $\mathcal{A}$ wins $G_{\mathcal{A}}^{\text{eval-binding}}(\lambda)$ is negligible in $\text{negl}(\lambda)$.
- **Polynomial Binding:** Consider a game $G_{\mathcal{A}}^{\text{poly-binding}}(\lambda)$. An adversary $\mathcal{A}$ takes public parameter λ as input and outputs $(P_1(X), P_2(X))$. It wins if $\text{Commit}(P_1(X)) = \text{Commit}(P_2(X))$ and $P_1(X) \neq P_2(X)$. We say that the PCS satisfies polynomial binding if for every PPT adversary $\mathcal{A}$, the probability that $\mathcal{A}$ wins $G_{\mathcal{A}}^{\text{poly-binding}}(\lambda)$ is negligible in $\text{negl}(\lambda)$.
- **Interpolation Binding:** Consider a game $G_{\mathcal{A}}^{\text{int-binding}}(\lambda)$. An adversary $\mathcal{A}$ takes public parameter (λ, t) as input and outputs $(\text{Com}, \{x_i, p_i, \pi_i\}_{i \in \{1, \dots, t+1\}})$. Then $\mathcal{A}$ interpolate $P(X)$ by $\text{interpolate}(\{(x_i, p_i)\}_{i \in \{1, \dots, t+1\}})$. It wins if (1) $x_i \neq x_j$ for all $i \neq j$, (2) $\text{Verify}(\text{Com}, x_i, p_i, \pi_i) = \text{True}$ for all $i \in \{1, \dots, t+1\}$ and (3) $\text{Com} \neq \text{Commit}(P(X))$. We say that the PCS satisfies interpolation binding if for every PPT adversary $\mathcal{A}$, the probability that $\mathcal{A}$ wins $G_{\mathcal{A}}^{\text{int-binding}}(\lambda)$ is negligible in $\text{negl}(\lambda)$.
- **Hiding:** Let $P_1(X)$ and $P_2(X)$ be any two polynomial of degree at most t . Consider a game $G_{\mathcal{A}}^{\text{hide}}(\lambda)$. An adversary $\mathcal{A}$ takes public parameter (λ, t) as input and outputs two challenge polynomial $P_1(X)$ and $P_2(X)$ of equal degree bound. The challenger samples a random bit $b \leftarrow \{0, 1\}$, computes $C^* \leftarrow \text{Commit}(P_b(x))$, and send C^* to $\mathcal{A}$. The adversary outputs a bit b' , and wins if $b' = b$. We say that

the PCS satisfies hiding if for every PPT adversary $\mathcal{A}$, its advantage $|\Pr[b' = b - \frac{1}{2}]|$ is negligible in $\text{negl}(\lambda)$.

- **Correctness:** If a commitment Com , an evaluation value p at x , and its proof π are generated honestly according to the protocol, then the verification $\text{Verify}(\text{Com}, (x, p), \pi)$ outputs True.

F. Zero-Knowledge Proof

Definition 6 (Schnorr Protocol). The Schnorr protocol [49], [50] is a well-known zero-knowledge proof protocol for proving knowledge of a discrete logarithm. It is widely used in identification schemes, where the prover convinces the verifier that it knows a secret x such that $h = g^x$, where g is a generator of a cyclic group $\mathbb{G}$, and h is the corresponding public value.

The original Schnorr protocol is interactive and consists of three main steps. First, the prover commits to a random value. Next, the verifier generates a random challenge. Finally, the prover computes a response by combining the challenge with the secret witness. This commit-challenge-response structure can incur considerable communication overhead, especially in distributed or asynchronous network environments.

Definition 7 (Fiat-Shamir Heuristic). The Fiat-Shamir heuristic [51] can transform an interactive protocol into a non-interactive proof. It replaces the verifier's challenge with a hash value derived from the prover's commitment and the public statement. This transformation significantly reduces communication complexity, as it allows the prover to generate the proof independently and enables the verifier to check it later without interaction. The prover computes the challenge as $c = H(g, h, u)$, where u denotes the prover's commitment. In the random oracle model, this heuristic provides a non-interactive proof that inherits the security properties of the original interactive protocol.

G. Asynchronous Byzantine Agreement

Definition 8 (Asynchronous Byzantine Agreement (ABA)). An ABA [35], [36] protocol for a set of nodes $\{1, \dots, n\}$, where each node holds an initial input and eventually outputs a value, while satisfying the following properties:

- **Agreement:** No two honest nodes output different values.
- **Validity:** If all honest nodes have the same input value, then no honest node will output a different value.
- **Termination:** All honest nodes will eventually output a value.

IV. FT-ACSS CONSTRUCTION

In this section, we present the proposed FT-ACSS protocol, including its polynomial structure, the adaptation of KZG protocol, the secret sharing phase, and the secret reconstruction phase, and finally provide the security analysis of FT-ACSS.

A. Secret Sharing Polynomial

Our polynomial construction supports packing $m \in [1, t+1]$ secrets into a single sharing process. Depending on the number of packed secrets, the construction is divided into the following three cases:

- 1) **Single Secret:** This case reduces to standard secret sharing. The secret s is embedded as the constant term of the polynomial, and the remaining t coefficients are sampled uniformly at random.

$$P(x) = s + r_1x^1 + r_2x^2 + \dots + r_{t-1}x^{t-1} + r_tx^t$$

- 2) **[2, t] Secrets:** Multiple secret embeddings are obtained either by interpolation or by assigning coefficients, while the remaining coefficients are sampled randomly.

$$P(x) = \underline{s_0} + \underline{s_1}x^1 + \underline{s_2}x^2 + \dots + r_{t-1}x^{t-1} + r_tx^t$$

- 3) **t + 1 Secrets:** This setting is closer to data encoding than to standard secret sharing. Since all coefficients or interpolation points are determined by the t + 1 secrets, no additional randomness remains, and thus the privacy guarantee differs from the other cases.

$$P(x) = \underline{s_0} + \underline{s_1}x^1 + \underline{s_2}x^2 + \dots + \underline{s_{t-1}}x^{t-1} + \underline{s_t}x^t$$

Our FT-ACSS scheme extends $P(x)$ into an asymmetric bivariate polynomial $\Phi(X, Y)$ [30], [36]. Its coefficients can be represented as a two-dimensional matrix M_Φ , as shown in Equation (1). The first row of the matrix is responsible for hiding secret, e.g. s_0 and s_1 . The vertical dimension is assigned a fixed degree of f , which serves to accelerate the sharing process. Furthermore, to strengthen the PCS-Binding property of the scheme, we introduce an additional fully random polynomial $\hat{\Phi}(X, Y)$ for sharing and verification. The polynomial $\hat{\Phi}$ has the same structure as Φ .

$$M_\Phi = \begin{Bmatrix} \underline{s_0}(r_{0,0}) & \underline{s_1}(r_{1,0}) & \cdots & r_{t-1,0} & r_{t,0} \\ r_{0,1} & r_{1,1} & \cdots & r_{t-1,1} & r_{t,1} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ r_{0,f-1} & r_{1,f-1} & \cdots & r_{t-1,f-1} & r_{t,f-1} \\ r_{0,f} & r_{1,f} & \cdots & r_{t-1,f} & r_{t,f} \end{Bmatrix} \quad (1)$$

This design reduces the impact of packet loss and malicious dealer behavior, thereby improving the robustness of our protocol and providing two main advantages:

- **Enhanced Security:** If $\Phi(X, Y)$ has degree d_1 in X and d_2 in Y , then reconstructing the original polynomial requires $(d_1 + 1) \times (d_2 + 1)$ valid shares. Compared with a univariate polynomial, this significantly increases the difficulty for an adversary to recover the secret polynomial.
- **Efficient and Robust Sharing:** In asynchronous networks, the dealer may fail to deliver all shares correctly. To address this issue, the Y -dimension allows nodes that have already received valid shares to help distribute shares to others. Each such node can use its local share polynomial to compute and forward the shares for other nodes. Once a node collects enough valid shares, it can reconstruct the corresponding share polynomial by interpolation.

B. Adaptation of KZG Commitments

The KZG commitment scheme is built on elliptic-curve bilinear pairings and relies on a trusted setup to generate the common reference string CRS. Our KZG adaptation is derived from the commitment framework of Bingo [36]. Specifically, we adopt a HashToCurve [52], [53] approach with randomized input to derive the auxiliary generators. This hash-based construction ensures that g and $\hat{g}$ are distributed independently, while allowing $\hat{g}$ to be publicly and efficiently verified by all parties.

```

CRS  $\leftarrow$  Setup( $crs', t, f$ )
//  $crs' = \{g \in \mathbb{G}_1, h \in \mathbb{G}_2, e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T\}$ 
 $\tau, seed \xleftarrow{\$} \mathbb{F}_q$ 
 $\hat{g} \xleftarrow{\$} HashToCurve(context || seed)$ 
 $CRS \leftarrow ((g, \hat{g}) \in \mathbb{G}_1, \{g^{\tau^i}, \hat{g}^{\tau^i}\}_{i \in \{0, \dots, t\}},$ 
 $(h, h^{\tau}) \in \mathbb{G}_2, \mathbb{G}_T, e, (t, f))$ 
return CRS

```

```

Comuni  $\leftarrow$  Commituni(CRS,  $\alpha(X), \hat{\alpha}(X)$ )
Com  $\leftarrow g^{\alpha(\tau)} \cdot \hat{g}^{\hat{\alpha}(\tau)}$ 
return Com

```

```

Combi  $\leftarrow$  Commitbi(CRS,  $\Phi(X, Y), \hat{\Phi}(X, Y)$ )
 $\phi_i(X) \leftarrow r_{0,i}X^0 + r_{1,i}X^1 + \dots + r_{t,i}X^t$ 
 $\hat{\phi}_i(X) \leftarrow \hat{r}_{0,i}X^0 + \hat{r}_{1,i}X^1 + \dots + \hat{r}_{t,i}X^t$ 
Com $\{\}$   $\leftarrow \{g^{\phi_i(\tau)} \cdot \hat{g}^{\hat{\phi}_i(\tau)}\}_{i=0}^f$ 
return Com $\{\dots\}$ 

```

```

CM  $\leftarrow$  DivideCom(CRS, Combi,  $n$ )
 $CM_i \leftarrow g^{\sum_{w=0}^f [\phi_w(\tau)]i^w} \cdot \hat{g}^{\sum_{w=0}^f [\hat{\phi}_w(\tau)]i^w}$ 
CM $\{\}$   $\leftarrow \{CM_i\}_{i \in [n]}$ 
return CM $\{\dots\}$ 

```

```

 $p, \hat{p}, \pi \leftarrow$  Eval(CRS,  $\alpha(X), \hat{\alpha}(X), x$ )
 $p \leftarrow \alpha(x), \hat{p} \leftarrow \hat{\alpha}(x)$ 
 $\theta(\tau) \leftarrow (\alpha(\tau) - p)/(\tau - x)$ 
 $\hat{\theta}(\tau) \leftarrow (\hat{\alpha}(\tau) - \hat{p})/(\tau - x)$ 
 $\pi \leftarrow g^{\theta(\tau)} \hat{g}^{\hat{\theta}(\tau)}$ 
return  $\pi$ 

```

```

True/False  $\leftarrow$  Verify(CRS, CM,  $(i, j), (p, \hat{p}), \pi$ )
 $CM_j \leftarrow CM(j)$ 
if  $e(CM_j \cdot g^{-p} \cdot \hat{g}^{-\hat{p}}, h) = e(\pi, h^{\tau-i})$  then return True
else return False

```

Fig. 2. Functions in the Flexible-Threshold Adaptation of Our KZG-Based Scheme.

The overview of our KZG-based scheme is shown in Figure.2. The detailed procedures are described as follows:

- $CRS \leftarrow Setup(crs', t, f)$: Based on the *PCS-binding*, our scheme additionally introduces a polynomial $\hat{\Phi}$ with fully random coefficients. To commit to $\hat{\Phi}$, an auxiliary generator $\hat{g} \in \mathbb{G}_1$ is required. A straightforward approach is to sample $a \xleftarrow{\$} \mathbb{F}_q$ and set $\hat{g} = g^a$. However, this construction creates a known discrete-log relation between g and $\hat{g}$, which is

undesirable in pairing-based commitment schemes and may weaken the soundness of the proof system. To avoid this issue, we derive $\hat{g}$ using *HashToCurve*.

Specifically, the trusted setup (of size $O(\lambda n)$ words) can realize through MPC or multi-party ceremony, which jointly generates two independent random values: a hidden trapdoor τ , used to construct the powers-of- τ CRS, and a public random *seed*, used to derive $\hat{g}$. The seed is published as part of the public parameters, and every node can verify $\hat{g}$. In this way, the CRS remains compatible with the original KZG procedure, while the auxiliary generator is publicly derivable and does not rely on any known algebraic relation to the original generator g . This design improves the security and verifiability.

- $Com_{uni} \leftarrow Commit_{uni}(CRS, \alpha(X), \hat{\alpha}(X))$: Using the additive homomorphic property of the commitment scheme, $Commit_{uni}$ computes the commitments to $\alpha(X)$ and $\hat{\alpha}(X)$ under the generators g and $\hat{g}$, respectively. It then returns their sum as Com .
- $Com_{bi} \leftarrow Commit_{bi}(CRS, \Phi(X, Y), \hat{\Phi}(X, Y))$: The coefficient matrix is interpreted row by row, where each row is treated as an independent univariate polynomial in X . A separate commitment is then generated for each row, resulting in a set of $f + 1$ polynomial commitments.
- $CM \leftarrow DivideCom(CRS, Com_{bi}, n)$: Upon receiving Com_{bi} , each party applies $DivideCom$ to decompose it into n sub-commitments, each corresponding to one party. Relying on the additive homomorphic property, $DivideCom$ derives for every party $i \in [n]$, enabling verification of the share associated with that party. The computation of the sub-commitments CM_i is described as follows:

$$\sum_{w=0}^f [g^{\phi_w(\tau)} \hat{g}^{\hat{\phi}_w(\tau)}] i^w = g^{\sum_{w=0}^f \phi_w(\tau) i^w} \cdot \hat{g}^{\sum_{w=0}^f \hat{\phi}_w(\tau) i^w}$$

Let $X = \tau$ and $Y = i$, we can obtain $\sum_{w=0}^f [\phi_w(\tau)] i^w = \Phi(\tau, i) = \alpha_i(\tau)$ and $\sum_{w=0}^f [\hat{\phi}_w(\tau)] i^w = \hat{\alpha}_i(\tau)$:

$$= \begin{bmatrix} r_{0,0}i^0 & r_{1,0}\tau^1i^0 & \dots & r_{t,0}\tau^ti^0 \\ \vdots & \vdots & \ddots & \vdots \\ r_{0,f-1}i^{f-1} & r_{1,f-1}\tau^1i^{f-1} & \dots & r_{t,f-1}\tau^ti^{f-1} \\ r_{0,f}i^f & r_{1,f}\tau^1i^f & \dots & r_{t,f}\tau^ti^f \end{bmatrix}$$

Thus, Similarly, $\hat{g}^{\sum_{w=0}^f [\hat{\phi}_w(\tau)] i^w} = \hat{g}^{\hat{\alpha}_i(\tau)}$. After receiving α' and $\hat{\alpha}'$, verifier can compute $Commit_{uni}(\alpha'(X), \hat{\alpha}'(X))$. Due to *PCS-binding*, the verification will succeed if and only if $\alpha = \alpha'$ and $\hat{\alpha} = \hat{\alpha}'$.

- $p, \hat{p}, \pi \leftarrow Eval(CRS, \alpha(X), \hat{\alpha}(X), x)$: This function can generate proofs for the shares provided to other parties. When computing $\theta(\tau)$, τ is required to calculate $\alpha(\tau)$ and $(\tau - x)$, but τ must not be revealed to external nodes. Thus, we need to use interpolation to compute $\theta(X)$. For node i , construct the point set $\mathcal{P}_i$ which contains $f + 1$ evaluation points of $\alpha_i(X)$, where the evaluation point i is excluded from $\mathcal{J}_i$:

$$\mathcal{J}_i \subseteq [n] \setminus \{i\}, |\mathcal{J}_i| = f + 1,$$

$$\mathcal{P}_i = \{(j, \alpha_i(j)) | j \in \mathcal{J}_i\}$$

Define the shifted point set $\mathcal{P}_{\theta_i}$:

$$\mathcal{P}_{\theta_i} = \{(j - i, \alpha_i(j) - \alpha_i(i)) | j \in \mathcal{J}_i\}$$

By interpolating $\mathcal{P}_{\theta_i}$, we can obtain $\theta_i(X)$:

$$\text{Interpolate}(\mathcal{P}_{\theta_i}) = \theta_i(X) = \frac{\alpha_i(X) - \alpha_i(i)}{X - i}$$

The same procedure applies to $\hat{\alpha}_i(X)$, yielding $\hat{\theta}_i(X)$. Finally, output a proof π :

$$\pi = g^{\theta_i(\tau)} \cdot \hat{g}^{\hat{\theta}_i(\tau)} = g^{(\frac{\alpha(\tau)-p}{\tau-i})} \cdot \hat{g}^{(\frac{\hat{\alpha}(\tau)-\hat{p}}{\tau-i})}$$

- True/False $\leftarrow \text{Verify}(\text{CRS}, \text{CM}, (i, j), (p, \hat{p}), \pi)$: This procedure checks the consistency of the claimed evaluations $(p, \hat{p})$ with respect to the j -th sub-commitment CM_j at point i . Based on the additive homomorphism, $\text{CM}_j \cdot g^{-p} \cdot \hat{g}^{-\hat{p}}$ satisfies the following equations:

$$\text{CM}_j \cdot g^{-p} \cdot \hat{g}^{-\hat{p}} = \left(g^{\frac{\alpha_j(\tau)-p}{\tau-i}} \cdot \hat{g}^{\frac{\hat{\alpha}_j(\tau)-\hat{p}}{\tau-i}} \right)^{\tau-i} = \pi^{\tau-i}$$

The verifier then checks whether the following bilinear pairing equation holds:

$$\begin{aligned} e(\text{CM}_j \cdot g^{-p} \cdot \hat{g}^{-\hat{p}}, h) &= e(\pi^{\tau-i}, h) \\ &= e(\pi, h)^{\tau-i} = e(\pi, h^{\tau-i}) \end{aligned}$$

By correctness and binding, the check succeeds only when the claimed values satisfy $p = \alpha_j(i)$ and $\hat{p} = \hat{\alpha}_j(i)$, except with negligible probability.

C. Overview of Our FT-ACSS Sharing Phase

During the sharing phase, the dealer first constructs bivariate asymmetric polynomials $\Phi(X, Y)$ and $\hat{\Phi}(X, Y)$ under the target reconstruction threshold $t \in [f, n - f]$, with the secret embedded in $\Phi(X, Y)$. Using the adapted KZG commitment scheme described above, the dealer further generates the corresponding commitments and opening proofs, enabling each parties to verify the correctness of its received share.

Algorithm 1 GenPolyandRbcCom

Require: $n, t, f, [s_0, \dots, s_k]$

- 1: $P(X) \leftarrow \text{Interpolate}([s_0, \dots, s_k])$
 - 2: uniformly sample coefficients from $\mathbb{F}_q$ to fill $\Phi(X, Y)$
 - 3: uniformly sample $\hat{\Phi}(X, Y)$ with degree t in X , f in Y
 - 4: $\text{Com}_{\text{bi}} \leftarrow \text{Commit}_{\text{bi}}(\text{CRS}, \Phi(X, Y), \hat{\Phi}(X, Y))$
 - 5: **RBC**(‘commitments’, Com_{bi})
 - 6: $i \in [n]$:
 - 7: $\alpha_i(X) \leftarrow \Phi(X, i), \hat{\alpha}_i(X) \leftarrow \hat{\Phi}(X, i)$
 - 8: send ‘poly_alpha’, $\alpha_i(X), \hat{\alpha}_i(X)$ to party i
-

As shown in Algorithm 1 and 2, the sharing phase of our FT-ACSS protocol consists of five main steps:

- **Preprocessing, Algorithm 1:** Given the total number of parties n , the reconstruction threshold t , the fault threshold f , and the input secrets $[s_0, \dots, s_k]$, the dealer first constructs the secret polynomial $\Phi(X, Y)$ and random polynomial $\hat{\Phi}(X, Y)$. Then, the dealer generate commitments to Φ and $\hat{\Phi}$, and broadcast to all parties through RBC [22], [41]. Due to the validity, agreement and totality guarantees of the

Algorithm 2 Sharing Phase (for node i)

Require: n, t, f

- 1: $\alpha_i \leftarrow \perp, \hat{\alpha}_i \leftarrow \perp, \text{accept}_{\alpha_i} \leftarrow \emptyset, \text{accept}_{\hat{\alpha}_i} \leftarrow \emptyset$
 - 2: $\beta_i \leftarrow \perp, \hat{\beta}_i \leftarrow \perp, \text{accept}_{\beta_i} \leftarrow \emptyset, \text{accept}_{\hat{\beta}_i} \leftarrow \emptyset$
 - 3: $\text{CM} \leftarrow \emptyset$
 - 4: **upon receiving** (‘commitments’, Com_{bi}) **from RBC**:
 - 5: $\text{CM} \leftarrow \text{DivideCom}(\text{Com}_{\text{bi}}, n)$
 - 6: **upon receiving** (‘poly_alpha’, $\alpha_i', \hat{\alpha}_i'$) **from dealer**:
 - 7: **upon** $\text{CM} \neq \emptyset$
 - 8: **if** $\alpha_i = \perp$ **and** $\text{Commit}_{\text{uni}}(\alpha_i', \hat{\alpha}_i') = \text{CM}_i$:
 - 9: $\alpha_i \leftarrow \alpha_i', \hat{\alpha}_i \leftarrow \hat{\alpha}_i'$
 - 10: **upon** $\alpha_i \neq \perp$:
 - 11: **For** $j \in [n]$ **do**
 - 12: $(\alpha_i(j), \hat{\alpha}_i(j), \pi_{\alpha_{i,j}}) \leftarrow \text{Eval}(\alpha_i, \hat{\alpha}_i, j)$
 - 13: send (‘sh_alpha’, $\alpha_i(j), \hat{\alpha}_i(j), \pi_{\alpha_{i,j}}$) to party j
 - 14: **upon receiving** (‘sh_alpha’, $\alpha_j(i), \hat{\alpha}_j(i), \pi_{\alpha_{j,i}}$) **from** j :
 - 15: **upon** $\text{CM} \neq \emptyset$
 - 16: **if** $|\text{accept}_{\beta_i}| \leq f$ **and** $|\text{accept}_{\hat{\beta}_i}| \leq f$:
 - 17: **if** $\text{Verify}(\text{CM}, (i, j), \alpha_j(i), \hat{\alpha}_j(i), \pi_{\alpha_{j,i}}) = \text{True}$:
 - 18: $\{\alpha_j(i) = \beta_i(j), \hat{\alpha}_j(i) = \hat{\beta}_i(j)\}$
 - 19: $\text{accept}_{\beta_i} \leftarrow \text{accept}_{\beta_i} \cup \{(j, \beta_i(j))\}$
 - 20: $\text{accept}_{\hat{\beta}_i} \leftarrow \text{accept}_{\hat{\beta}_i} \cup \{(j, \hat{\beta}_i(j))\}$
 - 21: **else if** $|\text{accept}_{\beta_i}| = |\text{accept}_{\hat{\beta}_i}| = f + 1$:
 - 22: $\beta_i \leftarrow \text{Interpolate}(\text{accept}_{\beta_i})$
 - 23: $\hat{\beta}_i \leftarrow \text{Interpolate}(\text{accept}_{\hat{\beta}_i})$
 - 24: **for each** $j \in [n]$:
 - 25: $(\beta_i(j), \hat{\beta}_i(j), \pi_{\beta_{i,j}}) \leftarrow \text{Eval}(\beta_i, \hat{\beta}_i, j)$
 - 26: send (‘sh_beta’, $\beta_i(j), \hat{\beta}_i(j), \pi_{\beta_{i,j}}$) to party j
 - 27: **Upon receiving** (‘sh_beta’, $\beta_j(i), \hat{\beta}_j(i), \pi_{\beta_{j,i}}$) **from** j :
 - 28: **upon** $\text{CM} \neq \emptyset$:
 - 29: **if** $\alpha_i = \perp$ **and** $\hat{\alpha}_i = \perp$:
 - 30: **if** $\text{Verify}(\text{CM}, (j, i), \beta_j(i), \hat{\beta}_j(i), \pi_{\beta_{j,i}}) = \text{True}$:
 - 31: $\{\beta_j(i) = \alpha_i(j), \hat{\beta}_j(i) = \hat{\alpha}_i(j)\}$
 - 32: $\text{accept}_{\alpha_i} \leftarrow \text{accept}_{\alpha_i} \cup \{(j, \alpha_i(j))\}$
 - 33: $\text{accept}_{\hat{\alpha}_i} \leftarrow \text{accept}_{\hat{\alpha}_i} \cup \{(j, \hat{\alpha}_i(j))\}$
 - 34: **if** $|\text{accept}_{\alpha_i}| = |\text{accept}_{\hat{\alpha}_i}| = t + 1$:
 - 35: $\alpha_i \leftarrow \text{Interpolate}(\text{accept}_{\alpha_i})$
 - 36: $\hat{\alpha}_i \leftarrow \text{Interpolate}(\text{accept}_{\hat{\alpha}_i})$
 - 37: **upon** $\alpha_i \neq \perp, \hat{\alpha}_i \neq \perp, \beta_i \neq \perp, \hat{\beta}_i \neq \perp$:
 - 38: send (‘done’) to all parties
 - 39: **Terminate**
-

RBC [22], [41], the commitment information is eventually delivered consistently to all honest parties. Subsequently, the dealer evaluates the bivariate polynomials at $Y = i$ for each $i \in [n]$, thereby obtaining n pairs of univariate polynomials $(\alpha(X), \hat{\alpha}(X))$ of degree t , which are then sent to the corresponding parties over authenticated point-to-point channels.

- **Dissemination of $(\alpha, \hat{\alpha})$, Algorithm 2 Line 4-13:** After receiving the global commitment Com from RBC, node i derives sub-commitments via DivideCom and uses them to verify $\alpha(X)$ and $\hat{\alpha}(X)$ sent by the dealer. Once accepted,

node i calculates $\alpha_i(j)$ and $\hat{\alpha}_i(j)$ for all $j \in [n]$, generates the corresponding proofs $\pi_{\alpha_i,j}$, and forwards the shares to the respective parties.

- **Reconstruction of $(\beta, \hat{\beta})$, Algorithm 2 Line 14-25:** Upon receiving a share message, node i verifies the evaluations against the sub-commitment CM, and valid shares are stored in the acceptance sets. When the size of the acceptance sets reaches $f + 1$, node i is able to reconstruct β and $\hat{\beta}$. It then calculates $\beta_i(j)$ and $\hat{\beta}_i(j)$ for all parties $j \in [n]$, computes the corresponding proofs $\pi_{\beta_i,j}$, and forwards the resulting values to the respective parties, thereby assisting them in recovering their α -polynomials.
- **Reconstruction of $(\alpha, \hat{\alpha})$, Algorithm 2 Line 26-36:** This stage follows the same pattern as the previous one. Node i receives and verifies the evaluation shares sent by other parties, and once $t + 1$ valid shares have been collected, it reconstructs α and $\hat{\alpha}$. After α , $\hat{\alpha}$, β and $\hat{\beta}$ have all been successfully recovered, node i broadcasts a done message to all parties.
- **Termination, Algorithm 2 Line 37-39:** Once node i has successfully reconstructed all four local polynomials and has received done messages from at least $n - f$ parties, the sharing phase is considered complete and node i terminates.

The key idea of the sharing phase is to accelerate dissemination through polynomial reconstruction. Once a subset of honest parties has obtained valid α and $\hat{\alpha}$, they can immediately assist other parties in reconstructing β and $\hat{\beta}$. Since the Y -dimension has degree f , as soon as $f + 1$ honest parties hold valid $\alpha(X)$ and $\hat{\alpha}(X)$, they already provide sufficient information for every party in the network to reconstruct its local $\beta(Y)$ and $\hat{\beta}(Y)$. In turn, once at least $t + 1$ valid shares disseminated through the reconstructed β -polynomials, all parties can reconstruct their local α -polynomials, and hence the entire network gains the ability to reconstruct the global $\Phi(X, Y)$ and $\hat{\Phi}(X, Y)$.

Therefore, even in an asynchronous network, it is sufficient for the dealer to deliver valid shares to $f + 1$ honest parties, after which the sharing process can proceed without further assistance from the dealer. This feature fundamentally overcomes the classical dealer-offline problem and constitutes the completeness of FT-ACSS.

D. Secret Reconstruction Phase

Algorithm 3 Reconstruction (for node i)

Require: t , CM

- 1: $\text{shares}_i \leftarrow \emptyset$
 - 2: $(\beta_i(0), \hat{\beta}_i(0), \pi_{\beta_i,0}) \leftarrow \text{Eval}(\beta_i, \hat{\beta}_i, 0)$
 - 3: send $\langle \text{'rec'}, \beta_i(0), \hat{\beta}_i(0), \pi_{\beta_i,0} \rangle$

 - 4: **upon receiving** $\langle \text{'rec'}, \beta_j(0), \hat{\beta}_j(0), \pi_{\beta_j,0} \rangle$ **from** j :
 - 5: **if** $\text{Verify}(\text{CM}, (j, 0), \beta_j(0), \hat{\beta}_j(0), \pi_{\beta_j,0}) = \text{True}$:
 $\{\beta_j(0) = \alpha_0(j)\}$
 - 6: $\text{share}_i \leftarrow \text{share}_i \cup \{(j, \alpha_0(j))\}$
 - 7: **if** $|\text{share}_i| = t + 1$ **then**
 - 8: $\alpha_0(X) \leftarrow \text{Interpolate}(\text{share}_i)$
 - 9: **Terminate**
-

We describe how to reconstruct our shared secret polynomial in Algorithm 3. After all nodes have completed sharing, the secret reconstruction phase can begin.

The set of secrets is hidden in the polynomial $\alpha_0(X)$. Parties use their β and $\hat{\beta}$ polynomials to compute $\beta_i(0)$, $\hat{\beta}_i(0)$ and proof $\pi_{\beta_i,0}$. Once the received secret shares reach $t + 1$, reconstruction can begin. Using interpolation, the polynomial $\alpha_0(X)$ is reconstructed. Subsequently, the original secret can be obtained.

E. Security Analysis of FT-ACSS

Lemma 1. If DL, q -SDH and ROM assumptions hold, then our KZG-based PCS satisfies evaluation binding, polynomial binding and interpolation binding (Definition 5).

Proof. (Evaluation binding) If an adversary can produce two distinct valid proofs for the same commitment at the same point, then either this induces two different valid proofs of the same hidden KZG commitment at the same point, contradicting the q -SDH-based binding of KZG, or it reveals the discrete-log relation between $(g, \hat{g})$ and commitment, contradicting the DL assumption. (Polynomial Binding) Since two distinct polynomials consistent with the same commitment must differ at some point, yielding two distinct valid proofs at that point. (Interpolation Binding) Since $d + 1$ valid proofs at distinct points uniquely determine a degree- d polynomial, and any non-uniqueness would contradict polynomial binding. $\square$

Lemma 2. If q -SDH and interpolation binding hold, then our KZG-based PCS in Fig.2 satisfies hiding and correctness.

Proof. Hiding follows because the random polynomial $\hat{\Phi}(X, Y)$ contributes an independent random masking in the exponent, making the commitment distribution independent of the secret polynomial. The correctness holds due to the uniqueness of the Lagrange interpolation. $\square$

Theorem 1. If DL, q -SDH and ROM assumptions hold, then our KZG-based PCS satisfies binding, hiding, and correctness, and is therefore secure.

Proof. By Lemma 1, our PCS satisfies evaluation binding, polynomial binding, and interpolation binding. By Lemma 2, it further satisfies hiding and correctness, where the random polynomial strengthens hiding while preserving correctness. Combining these properties with the security notions defined above, we conclude that our PCS is secure. $\square$

Lemma 3. The FT-ACSS protocol satisfies the Termination property.

Proof. If the dealer is honest, it first broadcasts the global commitment Com through RBC. By the termination and agreement properties of RBC, every honest party eventually receives same commitment. The dealer then sends to each honest party i its local polynomial pair $(\alpha_i, \hat{\alpha}_i)$. Since these values are consistent with the corresponding sub-commitment CM _{i} , every honest party accepts them after verification.

Once an honest party accepts $(\alpha_i, \hat{\alpha}_i)$, it evaluates them at $x = j$ for all $j \in \{1, \dots, n\}$, generates the corresponding proofs, and disseminates the shares. Since there are at least

$n - f$ honest parties and the Y -dimension has degree f , each honest party eventually receives at least $f + 1$ valid shares and can reconstruct $(\beta_i, \hat{\beta}_i)$. It then evaluates at all points and disseminates the resulting evaluations. Since $t < n - f$, each honest party eventually collects at least $t + 1$ valid evaluations and reconstructs $(\alpha_i, \hat{\alpha}_i)$. Consequently, every honest party terminates. $\square$

Lemma 4. The FT-ACSS protocol satisfies the Agreement property.

Proof. By RBC, all honest parties receive the same global commitment Com. Any dealer shares or evaluations accepted by an honest party must verify against the corresponding sub-commitment. By the binding property of the PCS, for each evaluation point there exists at most one proof consistent with the commitment. Therefore, all valid shares accepted by honest parties must lie on the same committed polynomial.

Based on the interpolation binding property, the final polynomials recovered by honest nodes are consistent with the same committed polynomials $(\Phi, \hat{\Phi})$. Therefore, FT-ACSS satisfies Agreement. $\square$

Lemma 5. The FT-ACSS protocol satisfies the Privacy property.

Proof. Consider any PPT adversary corrupting at most f parties. Its view consists of the public commitment information, the associated proofs, and the local shares received by the corrupted parties. Since the sharing polynomial has degree t in the X -dimension and $f < t + 1$, the shares observed by the adversary are insufficient to derive the original secrets. Fully random polynomial $\hat{\Phi}$ introduces a higher randomness. Therefore, conditioned on the corrupted parties' local outputs, the distribution of honest shares is independent of the secrets.

Moreover, based on the hiding property of the PCS, the public commitments and proofs reveal no additional information beyond the corrupted shares. Hence, FT-ACSS preserves privacy against the adversary corrupting at most f parties. $\square$

Lemma 6. The FT-ACSS protocol satisfies the Completeness property.

Proof. Once $f + 1$ honest parties have obtained valid $(\alpha, \hat{\alpha})$ -shares, they already provide enough information to reconstruct every $(\beta, \hat{\beta})$ for all parties, since the Y -dimension has degree f . After reconstructing $(\beta, \hat{\beta})$, honest parties disseminate the $(\alpha, \hat{\alpha})$ -evaluations needed by others. Since there are at least $n - f$ honest parties and $t < n - f$, every honest party eventually receives at least $t + 1$ valid evaluations and can reconstruct its own $(\alpha, \hat{\alpha})$.

An honest party reconstructs its local pair $(\beta, \hat{\beta})$, this means that at least $f + 1$ valid $(\alpha, \hat{\alpha})$ -sources already exist in the network. Since messages between honest parties are eventually delivered, all honest parties will eventually receive enough valid evaluations to reconstruct their own $(\beta, \hat{\beta})$, and subsequently $(\alpha, \hat{\alpha})$. After recovering all local polynomials, each honest party broadcasts 'done' message. Because at least $n - f$ parties are honest, every honest party eventually receives $n - f$ such messages and terminates. Therefore, FT-ACSS satisfies the completeness property. $\square$

Theorem 2. If the underlying commitment scheme is secure, then the FT-ACSS, as specified in Algorithms 1, 2 and 3, is an f -resilient packed ACSS for m secrets, for any $m \in [t]$.

Proof. First, the Correctness of commitment scheme ensures that all honestly generated commitments and evaluation proofs are accepted by honest parties, thereby supporting the successful execution of the sharing and reconstruction phases. Second, the Binding property ensures that no adversary can make two inconsistent shares simultaneously verify under the same commitment. Hence, all accepted evaluations must be consistent with unique committed polynomials. This uniqueness is also used in the proofs of Agreement. Third, the Hiding property guarantees that the published commitment does not leak any additional information about the embedded secrets beyond what is revealed by corrupted shares.

Therefore, if underlying commitment scheme is secure, and based on Lemma 6, it follows that FT-ACSS is secure in asynchronous networks $\square$

Lemma 7. The expected total communication cost of our FT-ACSS protocol is $O(\lambda n^2)$.

Proof. Each protocol message contains only a constant number of group elements, field elements, or succinct proofs, and therefore has size $O(\lambda)$. The introduction of RBC incurs an $O(n^2)$ communication cost for commitment dissemination. In the sharing phase, each party must send the corresponding evaluations and proofs to all other parties, and subsequently broadcast a 'done' signal upon completion, which also results in $O(n^2)$ communication. In the reconstruction phase, each party similarly sends its recovered share to all other parties. Combining all phases, the total communication cost of the FT-ACSS protocol is $O(\lambda n^2)$. $\square$

V. m -SCHNORR AND FT-ADKG CONSTRUCTION

In this section, we show how to use FT-ACSS to achieve an flexible threshold ADKG (FT-ADKG) protocol that has $O(\lambda n^3)$ communication complexity. Similar to regular DKG protocols, the protocol consists of three main phases: sharing, agreement and key derivation. In the sharing phase, all parties execute n parallel instances of FT-ACSS to distribute random shares to every node in the network. In the agreement phase, all parties are required to agree on a common set of nodes. In the key derivation phase, the shares provided by the agreed set are used to derive the final public-key and distributed private-key.

First, we employ the Validated Asynchronous Byzantine Agreement (VABA) protocol proposed in the Bingo [36] framework to ensure that all honest parties agree on a common valid candidate set in an asynchronous setting. Second, we adapt the Schnorr [49], [54] protocol to the proposed FT-ACSS setup, yielding m -Schnorr protocol capable of generating signatures for m secret shares in a single instance. Third, the shares contributed by the agreed candidates are aggregated to derive m distributed public keys and the corresponding secret-key shares. The generated keys are field elements $(sk \in \mathbb{F}_q)$. Moreover, owing to the flexible-threshold structure inherited

from FT-ACSS, this versatility enables setting up threshold signature schemes [1]–[6] for different uses.

A. Agreement Phase

VABA [36] is a validated asynchronous Byzantine agreement protocol that enables all honest parties to agree on the same externally valid value even in the presence of malicious parties. In our FT-ADKG construction, the VABA is used to ensure that all honest parties agree on a valid set of candidate dealers, whose contributions are then combined to derive the final public-key and private-key-shares. The VABA primitive adopted here has been proven to be adaptively secure [36], making it suitable for asynchronous environments in which corruptions may occur dynamically during protocol execution.

B. Non-interactive m -Schnorr Protocol

In the key-derivation phase, each party needs to reveal its public-key shares to the other parties. To prove the correctness of these shares, we construct a non-interactive m -Schnorr protocol adapted from the classical Schnorr proof [49]. While the standard Schnorr protocol proves knowledge of a single secret, our m -Schnorr proves knowledge of multiple witnesses associated with multiple public-key shares. Formally, the statement–witness pair satisfies the following relation:

$$\mathcal{R} := \{ (\{pk_i\}_{i=1}^m, \{z_i\}_{i=1}^m) | pk_i = g^{z_i}, \forall i \in \{1, \dots, m\} \}$$

The overview of our m -Schnorr protocol is shown in Figure.3. The detailed procedures are described as follows:

- $\text{CRS} \leftarrow \text{Sc.Setup}(crs')$: The function initializes the public parameters required by m -Schnorr, by deriving an auxiliary generator $\hat{g} \in \mathbb{G}_1$ through *HashToCurve*. The output $\text{CRS} = (g, \hat{g})$ is then used in subsequent proof generation and verification. To avoid extra setup cost, this seed generation can be incorporated into the same MPC or multi-party ceremony [55] as the KZG initialization.
- $pk, \hat{pk}, \pi, \hat{\pi} \leftarrow \text{Sc.Prove}(\text{CRS}, z, \hat{z})$: Given the secret-share vectors z and $\hat{z}$, the prover first derives the corresponding public-key share vectors pk and $\hat{pk}$ under the generators g and $\hat{g}$, respectively. It then samples fresh randomness r and $\hat{r}$ to construct the Schnorr commitments U and $\hat{U}$. To remove interaction, the protocol applies the Fiat-Shamir transform, deriving the challenge values C and $\hat{C}$ by hashing the ordered public-key share vectors together with the corresponding commitments. Using these challenges, the prover computes the responses R and $\hat{R}$, where each response encodes the entire share vector through powers of the challenge. Finally, it outputs the proof pair $(\pi, \hat{\pi})$. In this way, a single m -Schnorr execution authenticates an ordered batch of m public-key shares, rather than requiring m independent Schnorr transcripts.
- $\text{True/False} \leftarrow \text{Sc.Verify}(\text{CRS}, pk, \hat{pk}, \pi, \hat{\pi})$: This function verifies the validity of the public-key share vectors pk and $\hat{pk}$ together with the proof pair $(\pi, \hat{\pi})$. The verifier first recomputes the Fiat-Shamir [51] challenges from the ordered public-key share vectors and the corresponding commitments. It then checks the batched Schnorr verification equations under the generators g and $\hat{g}$. The proof

is accepted if and only if both recomputed challenges are consistent and both verification equations hold.

```

CRS  $\leftarrow$  Sc.Setup( $crs'$ )
//  $crs' = \{g \in \mathbb{G}_1\}$ 
 $seed \xleftarrow{\$} \mathbb{F}_q$ 
 $\hat{g} \xleftarrow{\$} \text{HashToCurve}(\text{context} || seed)$ 
 $\text{CRS} \leftarrow ((g, \hat{g}) \in \mathbb{G}_1)$ 
return CRS

 $pk, \hat{pk}, \pi, \hat{\pi} \leftarrow \text{Sc.Prove}(\text{CRS}, z, \hat{z})$ 
 $r, \hat{r} \xleftarrow{\$} \mathbb{F}_q$ 
 $U \leftarrow g^r, \hat{U} \leftarrow \hat{g}^{\hat{r}}$ 
 $pk \leftarrow \{g^{z_i}\}_{i=1}^m, \hat{pk} \leftarrow \{\hat{g}^{\hat{z}_i}\}_{i=1}^m$ 
 $C \leftarrow \mathcal{H}(g || pk_1 || pk_2 || \dots || pk_m, U)$ 
 $\hat{C} \leftarrow \mathcal{H}(\hat{g} || \hat{pk}_1 || \hat{pk}_2 || \dots || \hat{pk}_m, \hat{U})$ 
 $R \leftarrow r + \sum_{i=1}^m z_i \cdot C^i$ 
 $\hat{R} \leftarrow \hat{r} + \sum_{i=1}^m \hat{z}_i \cdot \hat{C}^i$ 
 $\pi \leftarrow (U, C, R)$ 
 $\hat{\pi} \leftarrow (\hat{U}, \hat{C}, \hat{R})$ 
return  $pk, \hat{pk}, \pi, \hat{\pi}$ 

True/False  $\leftarrow \text{Sc.Verify}(\text{CRS}, pk, \hat{pk}, \pi, \hat{\pi})$ 
if
 $C = \mathcal{H}(g || pk_1 || pk_2 || \dots || pk_m, U)$  and
 $\hat{C} = \mathcal{H}(\hat{g} || \hat{pk}_1 || \hat{pk}_2 || \dots || \hat{pk}_m, \hat{U})$  and
 $U = g^R \cdot \prod_{i=1}^m pk_i^{-C^i}$  and  $\hat{U} = \hat{g}^{\hat{R}} \cdot \prod_{i=1}^m \hat{pk}_i^{-\hat{C}^i}$ 
then return True
else return False

```

Fig. 3. The m -Schnorr Protocol for Batched Proof of Knowledge. In this algorithm, pk , $\hat{pk}$, z , and $\hat{z}$ are length- m vectors. For any indexed notation, such as pk_i or z_i , the subscript i denotes the i -th element of the corresponding vector.

The main advantage of m -Schnorr is that it authenticates an ordered batch of public-key shares within a single proof execution. Rather than invoking a separate non-interactive Schnorr proof for each share, m -Schnorr reduces the authentication overhead from m independent proofs to a constant-size proof transcript.

C. Key Derivation Phase

Algorithm 4 presents the key derivation phase, in which the parties verify public-key shares and derive the final public key. Specifically, the verification procedure combines a KZG-based PCS with the proposed m -Schnorr protocol. The PCS ensures consistency between the disclosed shares and the underlying committed polynomial, whereas the m -Schnorr provides efficient batched authentication for the corresponding public-key share vectors. This phase therefore ensures both correctness of the share verification and soundness of the final public-key derivation.

The key derivation phase of our FT-ADKG protocol consists of five main steps:

- **Variable Initialization, Line 1-2:** After the VABA terminates, the set of qualified dealers, denoted by dealers , is

Algorithm 4 Key Derivation Phase (for node i)**Require:** dealers, m

```

1: share  $\leftarrow \emptyset$ ,  $\text{CM}_{\text{dealers}} \leftarrow \emptyset$ ,  $\hat{g}' \leftarrow \text{CRS}_{\text{KZG}}.\hat{g}$ 
2: key_pairs  $\leftarrow \emptyset$ 


---


3: for  $l \in \{0, \dots, m-1\}$ :
4:   for  $d \in \text{dealers}$ :
5:      $z_d(l), \hat{z}_d(l), \pi_d(l) \leftarrow \text{Eval}(\alpha_d, \hat{\alpha}_d, l)$ 
6:      $\{z_d(l) = \alpha_d(l), \hat{z}_d(l) = \hat{\alpha}_d(l)\}$ 
7:      $\text{CM}_d \leftarrow \text{DivideCom}(\text{Com}_{\text{bi}}(d), n)$ 
8:      $z(l) \leftarrow \sum_{d \in \text{dealers}} z_d(l)$ 
9:      $\hat{z}(l) \leftarrow \sum_{d \in \text{dealers}} \hat{z}_d(l)$ 
10:     $\bar{\pi}(l) \leftarrow \prod_{d \in \text{dealers}} \pi_d(l)$ 
11:     $\text{CM}_{\text{dealers}}(l) \leftarrow \prod_{d \in \text{dealers}} \text{CM}_d(l)$ 


---


12:  $z \leftarrow \{z(l)\}_{l \in \{0, \dots, m-1\}}$ 
13:  $\hat{z} \leftarrow \{\hat{z}(l)\}_{l \in \{0, \dots, m-1\}}$ 
14:  $\bar{\pi} \leftarrow \{\bar{\pi}(l)\}_{l \in \{0, \dots, m-1\}}$ 
15:  $\hat{p}'_i \leftarrow \{\hat{g}'^{\hat{z}(l)}\}_{l \in \{0, \dots, m-1\}}$ 
16:  $pk_i, \hat{p}_i, \pi_i, \hat{\pi}_i \leftarrow \text{Sc.Prove}(\text{CRS}, z, \hat{z})$ 
17: send ('pk_share',  $pk_i, \hat{p}_i, \pi_i, \hat{\pi}_i$ ) to all parties


---


18: receiving ('pk_share',  $pk_j, \hat{p}_j, \pi_j, \hat{\pi}_j$ ) from  $j$ 
19: if  $\text{Sc.Verify}(pk_j, \hat{p}_j, \pi_j, \hat{\pi}_j) = \text{True}$ :
20:   for  $l \in \{0, \dots, m-1\}$  do
21:     if  $\text{Verify}(\text{CM}_{\text{dealers}}(l), (l, j), \hat{p}_j(l), \pi_j(l)) = \text{True}$ :
22:       share $_l \leftarrow \text{share}_l \cup \{(j, pk_j(l))\}$ 
23:       if  $|\text{share}_l| = t + 1$ :
24:          $pk(l) \leftarrow \text{InterpolateExp}(\text{share}_l)$ 
25:         output  $(z(l), pk(l))$ 
26: output key_pairs  $\leftarrow \{(z(l), pk(l))\}_{l \in \{0, \dots, m-1\}}$ 
27: Terminate

```

taken as input to this algorithm. The parameter m denotes the number of public-key to be derived in the current round. Since the m -Schnorr protocol introduces a new auxiliary generator, we use $\hat{g}'$ to denote the auxiliary generator contained in the KZG, i.e., $\text{CRS}_{\text{KZG}}.\hat{g}$, so as to distinguish it from the auxiliary generator used in m -Schnorr.

- **Private-share Evaluation and Proof Aggregation, Line 3-10:** For each $l \in \{0, \dots, m-1\}$, node i evaluates the polynomials $\alpha_d, \hat{\alpha}_d$ contributed by every dealer $d \in \text{dealers}$ at the point $x = l$ by invoking Eval , thereby obtaining the corresponding evaluations $z_d(l), \hat{z}_d(l)$, and the proof $\pi_d(l)$. At the same time, CM_d is generated to store the commitment vector associated with dealer d , which will later be used for verification. The node then aggregates all evaluations and proofs from the dealers to obtain $z(l), \hat{z}(l)$ and $\bar{\pi}(l)$. In addition, $\text{CM}_{\text{dealers}}(l)$ records the aggregated polynomial commitment at $x = l$, which is used in the subsequent verification of received public-key shares.
- **Public-key-share Generation and Proof Construction, Line 11-16:** After all private-share values have been computed, the node organizes them into the vectors z and $\hat{z}$, together with the aggregated proof vector $\bar{\pi}$. These values are then used as inputs to Sc.Prove to generate the public-key share vectors and the corresponding m -Schnorr proofs.

Finally, node i broadcasts its public-key share message to all other parties.

- **Verification of Public-key Shares, Line 17-21:** Upon receiving a message from node j , node i invokes Sc.Verify to authenticate the disclosed public-key share vectors. If the m -Schnorr verification succeeds, then for every $l \in \{0, \dots, m-1\}$, node i further runs the KZG-based PCS verification. Only when both checks succeed is the received share inserted into the set share.
- **Derivation of Final Public Keys, Line 22-26:** Once the size of share $_l$ reaches $t + 1$, node i can reconstruct the corresponding public key by InterpolateExp . The resulting public key $pk(l)$ is then paired with the local secret-share value $z(l)$. After this procedure is completed for all $l \in \{0, \dots, m-1\}$, the node outputs the derived key pairs key_pairs and terminates.

$\text{InterpolateExp}(\text{share}_l)$ follows the structure of standard Lagrange interpolation, but operates in the exponent with respect to the generator g . In particular, instead of interpolating scalar values directly, it combines the public-key shares as group elements weighted by the corresponding Lagrange coefficients.

$$\text{InterpolateExp}(\cdot) = g^{z(x)} = g^{z_0 + z_1 x + \dots + z_t x^t}$$

Thus, the l -th public key is $pk(l) = g^{z(l)} = g^{z_0 + z_1 l + \dots + z_t l^t}$. This allows the target public key to be reconstructed without explicitly recovering the underlying scalar secret.

D. Security Analysis

Lemma 8. Assume that $\mathbb{G}_1$ is a cyclic group of prime order q generated by g , and that the witness vector $z = (z_1, \dots, z_m)$ satisfies $pk_l = g^{z_l}$ for all $l \in \{1, \dots, m\}$. Then the interactive m -Schnorr protocol satisfies completeness.

Proof. For an honestly generated transcript, the prover computes $U = g^r$ and $R = r + \sum_{l=1}^m z_l \cdot C^l$. Substituting $pk_l = g^{z_l}$ into the verification equation gives below:

$$g^R \cdot \prod_{l=1}^m pk_l^{-C^l} = g^{r + \sum_{l=1}^m z_l \cdot C^l} \cdot \prod_{l=1}^m g^{-z_l C^l} = g^r = U.$$

Hence every honestly generated transcript is accepted, and the protocol satisfies completeness. $\square$

Lemma 9. Assume that $\mathbb{G}_1$ is a cyclic group of prime order q generated by g . Suppose one is given $m + 1$ accepting transcripts $(U, C^{(1)}, R^{(1)}), \dots, (U, C^{(m+1)}, R^{(m+1)})$ for the same public-key-share vector $pk = (pk_1, \dots, pk_m)$, where the commitment U is fixed and the challenges $C^{(1)}, \dots, C^{(m+1)}$ are pairwise distinct. Then one can efficiently extract a witness vector $z = (z_1, \dots, z_m)$ such that $pk_l = g^{z_l}$ for all $l \in \{1, \dots, m\}$. The m -Schnorr protocol satisfies special soundness.

Proof. Given $m + 1$ accepting transcripts with the same commitment U and pairwise distinct challenges, the corresponding challenge-response pairs uniquely define a degree- m polynomial:

$$F(X) = r^* + \sum_{l=1}^m z_l^* \cdot X^l.$$

On the other hand, since each transcript satisfies the verification equation, $U = g^u$ and $pk_l = g^{a_l}$ can yield:

$$R^{(j)} = u + \sum_{l=1}^m a_l^* \cdot (C^{(j)})^l, \forall j \in 1, \dots, m+1.$$

Hence all accepting transcripts also lie on the polynomial:

$$G(X) = u + \sum_{l=1}^m a_l^* \cdot (X^{(j)})^l.$$

Both $F(X)$ and $G(X)$ have degree at most m and agree on $m+1$ pairwise distinct points, so they must be identical. Therefore $z_l^* = a_l$ for all l , and thus $pk_l = g^{z_l^*}$. Hence a valid witness vector can be efficiently extracted, which proves special soundness. $\square$

Lemma 10. Assume that $\mathbb{G}_1$ is a cyclic group of prime order q generated by g , and that the challenge C is chosen uniformly from F_q . Then the m -Schnorr protocol is honest-verifier zero-knowledge.

Proof. Consider the simulator $\mathcal{S}$ that samples $C, R \xleftarrow{\$} F_q$ uniformly and sets

$$U = g^R \cdot \prod_{l=1}^m pk_l^{-C^l}.$$

The resulting transcript is (U, C, R) . By construction, this transcript always satisfies the verification, and is therefore accepting. The honest prover samples $r \xleftarrow{\$} F_q$ and computes

$$R = r + \sum_{l=1}^m z_l \cdot C^l.$$

For any fixed challenge C , the response R is uniformly distributed over F_q , since r is uniform. Moreover, once C and R are fixed, the corresponding commitment U is uniquely determined by the verification equation. Hence, the simulated transcript is distributed identically to a real one generated in an honest prover.

Therefore, the view of honest verifier can be perfectly simulated without access to the witness, which establishes the honest-verifier zero-knowledge property of the m -Schnorr protocol. $\square$

Theorem 3. If ROM is hold, then the non-interactive m -Schnorr transformed via the Fiat-Shamir Heuristic [51] yields a non-interactive argument of knowledge for the relation $\mathcal{R}$ (Sect.V-B).

Proof. Assume that $\mathcal{H}$ is modeled as a random oracle. The non-interactive protocol computes the challenge as $C = \mathcal{H}(pk, U)$, where pk denotes the ordered public-key-share vector and U is the commitment. Its verification equation is identical to that of the underlying challenge-response form. Therefore, completeness follows directly from Lemma 8.

To establish knowledge soundness in the ROM. Assume there exists a PPT adversary $\mathcal{A}$ that outputs a valid non-interactive proof for a statement pk with non-negligible probability. We construct an extractor $\mathcal{E}$ that runs $\mathcal{A}$ while answering its random-oracle queries consistently. Whenever $\mathcal{A}$ produces

a valid proof (U, R) , the extractor can identify the oracle query of $\mathcal{H}(pk, U)$ that determines the challenge used in the proof. It then rewinds $\mathcal{A}$ to that query and reprograms the oracle so that the same input (pk, U) is answered with a fresh challenge value. Repeating this process yields $m+1$ accepting transcripts for the same statement pk and commitment U , but with pairwise distinct challenges. By Lemma 9, these $m+1$ accepting transcripts suffice to extract a witness vector $z = (z_1, \dots, z_m)$ such that $pk_l = g^{z_l}$ for all $l \in \{1, \dots, m\}$.

Therefore, any adversary capable of producing a valid Fiat-Shamir proof can be transformed into an extractor for the corresponding witness vector. This establishes that the m -Schnorr is a non-interactive argument of knowledge in the ROM. $\square$

Theorem 4. Assume that the underlying VABA [36] protocol is correct, the KZG-based PCS is binding and evaluation-sound, and the proposed m -Schnorr protocol satisfies completeness, special soundness, and honest-verifier zero-knowledge, with its Fiat-Shamir transformation secure in the ROM. Then the Key Derivation Phase is secure. Consequently, the FT-ADKG protocol securely derives consistent distributed key pairs for all honest parties.

Proof. By correctness of VABA, all honest parties start the key-derivation phase from the same qualified dealer set. For each index l , every received public-key-share message must pass both m -Schnorr verification and KZG-based PCS verification before being inserted into share_l . By the soundness of m -Schnorr, an adversary cannot authenticate an invalid ordered public-key-share vector without a corresponding witness. By the binding and evaluation soundness of the KZG-based PCS, an adversary cannot provide shares inconsistent with the committed polynomial structure. Hence, only valid shares consistent with the underlying dealer commitments are accepted by honest parties.

Once $|\text{share}_l| \geq t+1$, correctness and uniqueness of interpolation imply that all honest parties reconstruct the same public key at index l . Repeating this argument for all $l \in \{0, \dots, m-1\}$ shows that the entire key-pair is derived consistently. Therefore, the Key Derivation Phase is secure. Since this phase is the final step that transforms verified shares into distributed key pairs, and all previous protocol components already satisfy their required security properties, the overall FT-ADKG protocol is secure. $\square$

Lemma 11. The expected total communication cost of our FT-ADKG protocol is $O(\lambda n^3)$.

Proof. Each m -Schnorr transcript contains only a constant number of group elements and scalars, and therefore has size $O(\lambda)$. In the Key Derivation Phase, every party broadcasts one authenticated public-key-share message to all other parties, which yields $O(n^2)$ message transmissions. Each such message has size $O(\lambda)$, this phase contributes $O(\lambda n^2)$ communication. The VABA protocol from Bingo [36] has cost $O(\lambda n^3)$. Since FT-ADKG consists of $O(n)$ FT-ACSS instances. Therefore, the overall communication complexity of FT-ADKG is $O(\lambda n^3)$.

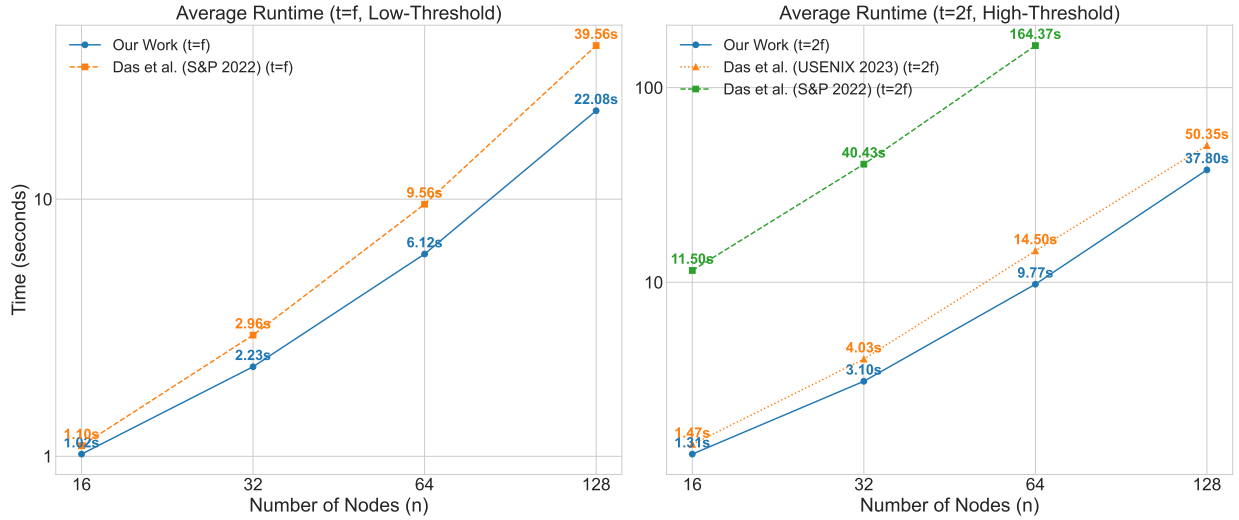


Fig. 4. Performance comparison of our FT-ADKG protocol against Das et al. [34]. The evaluation is split by threshold configuration: low-threshold ($t = f$) and high-threshold ($t = 2f$).

An additional advantage of FT-ADKG is its support for batched key derivation. Even when m public/private key pairs are generated simultaneously, the only component affected by m is the proof size, which still remains $O(\lambda)$. Hence, the protocol avoids a linear blow-up in communication with respect to the number of derived key pairs. This property is particularly beneficial in long-term system operation, where the amortized cost per key pair can be substantially reduced. $\square$

VI. IMPLEMENTATION AND EVALUATION

A. Implementation

We implemented a prototype of our protocol suite in Rust (https://github.com/MuZeRoFso/adkg_zk_lib). As our protocols rely on elliptic curve pairing properties, we selected the BLS12-381 [56] curve for all underlying cryptographic operations. All hash functions required by the protocols were instantiated using SHA-256.

We run all nodes on Amazon Web Services (AWS) t3a.medium virtual machines (VM) with one node per VM, across eight regions: Hong Kong, Tokyo, Singapore, Canada, London, Ireland, N. Virginia, and Oregon. A peer-to-peer network is established in which all nodes are pair-wise connected. Each VM was equipped with two vCPUs, 4GB of RAM, and ran Linux Ubuntu Server 22.04.

B. Evaluation

The evaluation demonstrates FT-ADKG's runtime performance. For the $n \geq 3f + 1$ setting, we evaluated our ADKG implementation across varying network scales ($n = \{16, 32, 64, 128\}$) and two distinct reconstruction thresholds: a low-threshold ($t = f$) and a high-threshold ($t = 2f$). We benchmarked our FT-ADKG against the current state-of-the-art ADKG protocols from Das et al. [34], [35], which represent the leading solutions for both low-threshold and high-threshold configurations. As shown in Fig. 4, our protocol demonstrates significant efficiency, and exhibits a particularly pronounced

performance advantage in the high-threshold ($t = 2f$) configuration.

VII. CONCLUSION

In this paper, we propose and thoroughly analyze a novel ACSS protocol that features flexible thresholds and the ability to share multiple secrets in a single round. In constructing the secret-sharing polynomial, we use interpolation to convert $m + 1$ secrets into a degree- m polynomial and then randomly select sufficient coefficients from a finite field to expand the polynomial into an asymmetric bivariate polynomial. Additionally, we extend the traditional Schnorr protocol into a non-interactive m -Schnorr protocol, with the aim of generating proofs for multiple public key shares. We construct an asynchronous distributed key generation protocol that supports flexible thresholds and can agree on multiple public keys in a single execution.

In an asynchronous network of n nodes, the communication complexity of our FT-ACSS protocol is $O(\lambda n^2)$. When extended to ADKG, our protocol achieves a communication complexity of $O(\lambda n^3)$ even in the worst case. Overall, our protocol is based on the q -SDH and ROM assumptions, requires PKI. We implemented the FT-ACSS and non-interactive m -Schnorr protocols in Rust.

REFERENCES

- [1] A. Boldyreva, "Threshold signatures, multisignatures and blind signatures based on the gap-diffie-hellman-group signature scheme," in *Public Key Cryptography — PKC 2003*, Y. G. Desmedt, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, pp. 31–46.
- [2] R. Gennaro, S. Jarecki, H. Krawczyk, and T. Rabin, "Secure distributed key generation for discrete-log based cryptosystems," *Journal of Cryptology*, vol. 20, pp. 51–83, 2007.
- [3] Y. G. Desmedt, "Threshold cryptography," *European Transactions on Telecommunications*, vol. 5, no. 4, pp. 449–458, 1994. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/ett.4460050407>
- [4] E. Kokoris-Kogias, E. C. Alp, L. Gasser, P. Jovanovic, E. Syta, and B. Ford, "CALYPSO: private data management for decentralized ledgers," *Proc. VLDB Endow.*, vol. 14, no. 4, pp. 586–599, 2020. [Online]. Available: <http://www.vldb.org/pvldb/vol14/p586-kogias.pdf>

- [5] C. Cachin, K. Kursawe, A. Lysyanskaya, and R. Stroh, "Asynchronous verifiable secret sharing and proactive cryptosystems," in *Proceedings of the 9th ACM Conference on Computer and Communications Security*, ser. CCS '02. New York, NY, USA: Association for Computing Machinery, 2002, p. 88–97. [Online]. Available: <https://doi.org/10.1145/586110.586124>
- [6] K. Gurkan, P. Jovanovic, M. Maller, S. Meiklejohn, G. Stern, and A. Tomescu, "Aggregatable distributed key generation," in *Advances in Cryptology – EUROCRYPT 2021*, A. Canteaut and F.-X. Standaert, Eds. Cham: Springer International Publishing, 2021, pp. 147–176.
- [7] A. Spiegelman, N. Girdharan, A. Sonnino, and L. Kokoris-Kogias, "Bullshark: DAG BFT protocols made practical," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7–11, 2022*, H. Yin, A. Stavrou, C. Cremers, and E. Shi, Eds. ACM, 2022, pp. 2705–2718. [Online]. Available: <https://doi.org/10.1145/3548606.3559361>
- [8] G. Danezis, L. Kokoris-Kogias, A. Sonnino, and A. Spiegelman, "Narwhal and tusk: a dag-based mempool and efficient BFT consensus," in *EuroSys '22: Seventeenth European Conference on Computer Systems, Rennes, France, April 5 - 8, 2022*, Y. Bromberg, A. Kermerrec, and C. Kozyrakis, Eds. ACM, 2022, pp. 34–50. [Online]. Available: <https://doi.org/10.1145/3492321.3519594>
- [9] R. Gelashvili, L. Kokoris-Kogias, A. Sonnino, A. Spiegelman, and Z. Xiang, "Jolteon and ditto: Network-adaptive efficient consensus with asynchronous fallback," in *Financial Cryptography and Data Security - 26th International Conference, FC 2022, Grenada, May 2–6, 2022, Revised Selected Papers*, ser. Lecture Notes in Computer Science, I. Eyal and J. A. Garay, Eds. Springer, 2022, pp. 296–315. [Online]. Available: https://doi.org/10.1007/978-3-031-18283-9_14
- [10] I. Keidar, E. Kokoris-Kogias, O. Naor, and A. Spiegelman, "All you need is DAG," in *PODC '21: ACM Symposium on Principles of Distributed Computing, Virtual Event, Italy, July 26–30, 2021*, A. Miller, K. Censor-Hillel, and J. H. Korhonen, Eds. ACM, 2021, pp. 165–175. [Online]. Available: <https://doi.org/10.1145/3465084.3467905>
- [11] G. Zhang, F. Pan, Y. Mao, S. Tjjanic, M. Dang'ana, S. Motepalli, S. Zhang, and H.-A. Jacobsen, "Reaching consensus in the byzantine empire: A comprehensive review of bft consensus algorithms," *ACM Comput. Surv.*, vol. 56, no. 5, Jan. 2024. [Online]. Available: <https://doi.org/10.1145/3636553>
- [12] R. Hao, X. Dai, and X. Xie, "Doppel: A BFT consensus algorithm for cyber-physical systems with low latency," *J. Syst. Archit.*, vol. 148, p. 103087, 2024. [Online]. Available: <https://doi.org/10.1016/j.sysarc.2024.103087>
- [13] R. Gennaro, S. Goldfeder, and A. Narayanan, "Threshold-optimal DSA/ECDSA signatures and an application to bitcoin wallet security," in *Applied Cryptography and Network Security - 14th International Conference, ACNS 2016, Guildford, UK, June 19–22, 2016. Proceedings*, ser. Lecture Notes in Computer Science, M. Manulis, A. Sadehgi, and S. A. Schneider, Eds. Springer, 2016, pp. 156–174. [Online]. Available: https://doi.org/10.1007/978-3-319-39555-5_9
- [14] I. M. Rodiana, B. Rahardjo, and A. I. W., "Design of a public key infrastructure-based single ballot e-voting system," in *2018 International Conference on Information Technology Systems and Innovation (ICITSI)*, 2018, pp. 6–9.
- [15] Y. Yang, B. Li, Q. Wu, B. Qin, Q. Wang, S. Xiong, and W. Susilo, "Randflash: Breaking the quadratic barrier in large-scale distributed randomness beacons," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 4710–4725, 2025.
- [16] W. Alnahari and M. T. Quasim, "Authentication of iot device and iot server using security key," in *2021 International Congress of Advanced Technology and Engineering (ICOTEN)*, 2021, pp. 1–9.
- [17] S. Ramesh and M. Govindarasu, "An efficient framework for privacy-preserving computations on encrypted iot data," *IEEE Internet of Things Journal*, vol. 7, no. 9, pp. 8700–8708, 2020.
- [18] J. Liu, Y. Du, K. Yang, J. Wu, Y. Wang, X. Hu, Z. Wang, Y. Liu, P. Sun, A. Boukerche, and V. C. M. Leung, "Edge-cloud collaborative computing on distributed intelligence and model optimization: A survey," *IEEE Commun. Surv. Tutorials*, vol. 28, pp. 5049–5080, 2026. [Online]. Available: <https://doi.org/10.1109/COMST.2026.3669216>
- [19] A. Shamir, "How to share a secret," *Commun. ACM*, vol. 22, no. 11, p. 612–613, Nov. 1979. [Online]. Available: <https://doi.org/10.1145/359168.359176>
- [20] G. R. Blakley, "Safeguarding cryptographic keys," in *1979 International Workshop on Managing Requirements Knowledge, MARK 1979, New York, NY, USA, June 4–7, 1979*. IEEE, 1979, pp. 313–318. [Online]. Available: <https://doi.org/10.1109/MARK.1979.8817296>
- [21] B. Chor, S. Goldwasser, S. Micali, and B. Awerbuch, "Verifiable secret sharing and achieving simultaneity in the presence of faults (extended abstract)," in *26th Annual Symposium on Foundations of Computer Science, Portland, Oregon, USA, 21–23 October 1985*. IEEE Computer Society, 1985, pp. 383–395. [Online]. Available: <https://doi.org/10.1109/SFCS.1985.64>
- [22] S. Das, Z. Xiang, and L. Ren, "Asynchronous data dissemination and its applications," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 2705–2721. [Online]. Available: <https://doi.org/10.1145/3460120.3484808>
- [23] T. Yurek, L. Luo, J. Fairuze, A. Kate, and A. Miller, "hbacss: How to robustly share many secrets," in *29th Annual Network and Distributed System Security Symposium, NDSS 2022, San Diego, California, USA, April 24–28, 2022*. The Internet Society, 2022. [Online]. Available: <https://www.ndss-symposium.org/ndss-paper/auto-draft-245/>
- [24] M. K. Franklin and M. Yung, "Communication complexity of secure computation (extended abstract)," in *Proceedings of the 24th Annual ACM Symposium on Theory of Computing, May 4–6, 1992, Victoria, British Columbia, Canada*, S. R. Kosaraju, M. Fellows, A. Wigderson, and J. A. Ellis, Eds. ACM, 1992, pp. 699–710. [Online]. Available: <https://doi.org/10.1145/129712.129780>
- [25] T. P. Pedersen, "A threshold cryptosystem without a trusted party," in *Advances in Cryptology – EUROCRYPT '91*, D. W. Davies, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 1991, pp. 522–526.
- [26] P. Feldman, "A practical scheme for non-interactive verifiable secret sharing," in *28th Annual Symposium on Foundations of Computer Science (sfcs 1987)*, 1987, pp. 427–438.
- [27] R. Canetti, R. Gennaro, S. Jarecki, H. Krawczyk, and T. Rabin, "Adaptive security for threshold cryptosystems," in *Advances in Cryptology – CRYPTO '99*, M. Wiener, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 1999, pp. 98–116.
- [28] W. Neji, K. Bliedch, and N. Ben Rajeb, "Distributed key generation protocol with a new complaint management strategy," *Security and Communication Networks*, vol. 9, no. 17, pp. 4585–4595, 2016. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/sec.1651>
- [29] N. Shrestha, A. Bhat, A. Kate, and K. Nayak, "Synchronous distributed key generation without broadcasts," *Cryptology ePrint Archive*, Paper 2021/1635, 2021. [Online]. Available: <https://eprint.iacr.org/2021/1635>
- [30] E. Kokoris Kogias, D. Malkhi, and A. Spiegelman, "Asynchronous distributed key generation for computationally-secure randomness, consensus, and threshold signatures," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 1751–1767. [Online]. Available: <https://doi.org/10.1145/3372297.3423364>
- [31] I. Abraham, P. Jovanovic, M. Maller, S. Meiklejohn, G. Stern, and A. Tomescu, "Reaching consensus for asynchronous distributed key generation," in *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing*, ser. PODC'21. New York, NY, USA: Association for Computing Machinery, 2021, p. 363–373. [Online]. Available: <https://doi.org/10.1145/3465084.3467914>
- [32] J. Doerner, Y. Kondi, E. Lee, and A. Shelat, "Threshold ecDSA from ecDSA assumptions: The multiparty case," in *2019 IEEE Symposium on Security and Privacy (SP)*, 2019, pp. 1051–1066.
- [33] J. Groth and V. Shoup, "Design and analysis of a distributed ECDSA signing service," *Cryptology ePrint Archive*, Paper 2022/506, 2022. [Online]. Available: <https://eprint.iacr.org/2022/506>
- [34] S. Das, T. Yurek, Z. Xiang, A. Miller, L. Kokoris-Kogias, and L. Ren, "Practical asynchronous distributed key generation," in *43rd IEEE Symposium on Security and Privacy, SP 2022, San Francisco, CA, USA, May 22–26, 2022*. IEEE, 2022, pp. 2518–2534. [Online]. Available: <https://doi.org/10.1109/SP46214.2022.9833584>
- [35] S. Das, Z. Xiang, L. Kokoris-Kogias, and L. Ren, "Practical asynchronous high-threshold distributed key generation and distributed polynomial sampling," in *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, CA: USENIX Association, Aug. 2023, pp. 5359–5376. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity23/presentation/das>
- [36] I. Abraham, P. Jovanovic, M. Maller, S. Meiklejohn, and G. Stern, "Bingo: Adaptivity and asynchrony in verifiable secret sharing and distributed key generation," in *Advances in Cryptology – CRYPTO 2023*, H. Handschuh and A. Lysyanskaya, Eds. Cham: Springer Nature Switzerland, 2023, pp. 39–70.
- [37] R. Bacho, J. Loss, G. Stern, and B. Wagner, "Harts: High-threshold, adaptively secure, and robust threshold schnorr signatures," in *Advances*

- in *Cryptology – ASIACRYPT 2024*, K.-M. Chung and Y. Sasaki, Eds. Singapore: Springer Nature Singapore, 2025, pp. 104–140.
- [38] R. Canetti and T. Rabin, “Fast asynchronous byzantine agreement with optimal resilience,” in *Proceedings of the Twenty-Fifth Annual ACM Symposium on Theory of Computing, May 16-18, 1993, San Diego, CA, USA*, S. R. Kosaraju, D. S. Johnson, and A. Aggarwal, Eds. ACM, 1993, pp. 42–51. [Online]. Available: <https://doi.org/10.1145/167088.167105>
- [39] M. Ben-Or, R. Canetti, and O. Goldreich, “Asynchronous secure computation,” in *Proceedings of the Twenty-Fifth Annual ACM Symposium on Theory of Computing, May 16-18, 1993, San Diego, CA, USA*, S. R. Kosaraju, D. S. Johnson, and A. Aggarwal, Eds. ACM, 1993, pp. 52–61. [Online]. Available: <https://doi.org/10.1145/167088.167109>
- [40] N. AlHaddad, M. Varia, and H. Zhang, “High-threshold avss with optimal communication complexity,” in *Financial Cryptography and Data Security*, N. Borisov and C. Diaz, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2021, pp. 479–498.
- [41] G. Bracha, “Asynchronous byzantine agreement protocols,” *Information and Computation*, vol. 75, no. 2, pp. 130–143, 1987. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/089054018790054X>
- [42] Y. Gao, Y. Lu, Z. Lu, Q. Tang, J. Xu, and Z. Zhang, “Efficient asynchronous byzantine agreement without private setups,” in *42nd IEEE International Conference on Distributed Computing Systems, ICDCS 2022, Bologna, Italy, July 10-13, 2022*. IEEE, 2022, pp. 246–257. [Online]. Available: <https://doi.org/10.1109/ICDCS54860.2022.00032>
- [43] A. Kate, G. M. Zaverucha, and I. Goldberg, “Constant-size commitments to polynomials and their applications,” in *Advances in Cryptology - ASIACRYPT 2010*, M. Abe, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 177–194.
- [44] M. Backes, A. Datta, and A. Kate, “Asynchronous computational vss with reduced communication complexity,” in *Topics in Cryptology – CT-RSA 2013*, E. Dawson, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 259–276.
- [45] V. Shoup and N. P. Smart, “Lightweight asynchronous verifiable secret sharing with optimal resilience,” *J. Cryptol.*, vol. 37, no. 3, p. 27, 2024. [Online]. Available: <https://doi.org/10.1007/s00145-024-09505-6>
- [46] N. Alhaddad, M. Varia, and Z. Yang, “Haven++: Batched and packed dual-threshold asynchronous complete secret sharing with applications,” *Cryptology ePrint Archive*, Paper 2024/326, 2024. [Online]. Available: <https://eprint.iacr.org/2024/326>
- [47] T. P. Pedersen, “Non-interactive and information-theoretic secure verifiable secret sharing,” in *Advances in Cryptology — CRYPTO ’91*, J. Feigenbaum, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 1992, pp. 129–140.
- [48] S. Goldwasser, Y. T. Kalai, and G. N. Rothblum, “Delegating computation: Interactive proofs for muggles,” *J. ACM*, vol. 62, no. 4, sep 2015. [Online]. Available: <https://doi.org/10.1145/2699436>
- [49] C. P. Schnorr, “Efficient identification and signatures for smart cards,” in *Advances in Cryptology — CRYPTO’ 89 Proceedings*, G. Brassard, Ed. New York, NY: Springer New York, 1990, pp. 239–252.
- [50] C.-P. Schnorr, “Efficient signature generation by smart cards,” *Journal of cryptology*, vol. 4, pp. 161–174, 1991.
- [51] A. Fiat and A. Shamir, “How to prove yourself: practical solutions to identification and signature problems,” in *Proceedings on Advances in Cryptology—CRYPTO ’86*. Berlin, Heidelberg: Springer-Verlag, 1987, p. 186–194.
- [52] T. Icart, “How to hash into elliptic curves,” in *Advances in Cryptology - CRYPTO 2009*, S. Halevi, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 303–316.
- [53] S. Chatterjee, D. Hankerson, E. Knapp, and A. Menezes, “Comparing two pairing-based aggregate signature schemes,” *Designs, Codes and Cryptography*, vol. 55, pp. 141–167, 2010.
- [54] R. Gennaro, D. Leigh, R. Sundaram, and W. S. Yee, “Batching schnorr identification scheme with applications to privacy-preserving authorization and low-bandwidth communication devices,” in *Advances in Cryptology - ASIACRYPT 2004, 10th International Conference on the Theory and Application of Cryptology and Information Security, Jeju Island, Korea, December 5-9, 2004, Proceedings*, ser. Lecture Notes in Computer Science, P. J. Lee, Ed. Springer, 2004, pp. 276–292. [Online]. Available: https://doi.org/10.1007/978-3-540-30539-2_20
- [55] S. Bowe, A. Gabizon, and I. Miers, “Scalable multi-party computation for zk-snark parameters in the random beacon model,” *IACR Cryptol. ePrint Arch.*, vol. 2017, p. 1050, 2017. [Online]. Available: <http://eprint.iacr.org/2017/1050>
- [56] S. Bowe, J. Grigg, and M. Lodder, “bls12_381_plus: Implementation of the bls12-381 pairing-friendly elliptic curve construction,” https://github.com/mikelodder7/bls12_381_plus, 2024. [Online]. Available: https://github.com/mikelodder7/bls12_381_plus